

Under Pressure: The Effects of Iteration Lengths on Agile Software Development Performance

Project Management Journal
Vol. 49(6) 78–102
© 2018 Project Management Institute, Inc.
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/8756972818802714
journals.sagepub.com/home/pmj


Kim E. van Oorschot¹, Kishore Sengupta², and Luk N. Van Wassenhove³

Abstract

The iterative cycles used in agile software development create a series of deadlines for project teams. We investigate the optimal iteration length under behavioral conditions for agile software projects. Based on system dynamics simulation, we argue that both very long (as in sequential development) and very short (monthly) iterative cycles create huge and disruptive fluctuations in schedule pressure experienced by the team. These lead to poor performance in terms of quality, as well as costs and time compared with optimal iterative cycles of two to three months. The monthly cycles used in many agile practices might be too short to ensure optimal performance.

Keywords

agile software development, deadlines, schedule pressure, simulation modeling, system dynamics

Introduction

Agile software development approaches are increasingly being adopted for the development of a wide array of products—from mobile phones to automobiles, healthcare, and financial services. These approaches are lightweight processes that employ short iterative cycles; actively involve users to establish, prioritize, and verify requirements; and rely on a team's tacit knowledge as opposed to documentation (Boehm & Turner, 2005, p. 32). These short iterative cycles create a series of deadlines; consequently, a sense of urgency or schedule pressure may be experienced right from the start of the project as opposed to near the end as in traditional methods.

The literature is full of support for agile software development approaches (Austin & Devin, 2009; Conboy, 2009; Zellkowitz, Wallace, & Binkley, 2003), but what are the practices in terms of the length and number of iterative cycles? These practices seem to be dominated by a normative perspective. Often a fixed iteration length of one month, or 20 working days, is recommended (Larman, 2006; Schatz & Abdelshafi, 2005), but these recommendations are made without analyzing how monthly deadlines influence schedule pressure, team behavior, and project management performance, despite the popularity and importance of agile approaches. The research on teams facing schedule pressure has yielded mixed results (Maruping, Venkatesh, Thatcher, & Patel, 2015). There are both positive (Gardner, 2012; Pearsall,

Ellis, & Stein, 2009; Rosso, 2014) and negative (Perlow, 1999; Zellmer-Bruhn, 2003) consequences related to deadlines for team behavior and project management performance. Extant literature offers, at best, a very limited understanding of the mechanisms through which agile approaches—such as 20-working-day intervals—affect team behavior (Maruping, Venkatesh, & Agarwal, 2009) and thereby project management performance (Conboy, 2009). This constitutes a significant gap in the agile literature that claims positive effects of iterative development, although we know from the literature on schedule pressure that negative effects can also occur. It is unclear which effects of schedule pressure outweigh the others and what their cumulative effect is when projects need to deliver incremental parts at consecutive deadlines rather than only one at the end.

In this study, we analyze how different iteration lengths (i.e., level of agility or the number of deadlines) in a project affect schedule pressure and, consequently, how schedule pressure influences the behavior of the team over time and project

¹ BI Norwegian Business School, Oslo, Norway

² Cambridge Judge Business School, Cambridge, United Kingdom

³ INSEAD Fontainebleau, France

Corresponding Author:

Kim E. van Oorschot, BI Norwegian Business School, Oslo, Norway.
Email: kim.v.oorschot@bi.no

management performance in terms of quality, costs, and time. As such, we examine if there is an optimal length of iterative cycles and, if so, what this optimal length is. We perform our analysis through simulation using a widely validated systems dynamics model based on theory and data from an actual software development project.

The contribution of our study is that we develop one feedback model that combines the technical aspect of agile development (multiple iterations, errors, quality, costs, time) with the human factors underlying such development (experience, exhaustion, turnover), including both the positive and negative effects that schedule pressure can have on these factors. Most organizations completely ignore these human factors (Ply, Moore, Williams, & Thatcher, 2012). Our study goes beyond these individual variables, connects them, and as such provides an explanation of the behavior of project teams under specific iteration lengths (and therefore, schedule pressure). The use of such a model enables us to formulate insights that relate iteration length (level of agility) with schedule pressure and, as such, the behavior of project teams, and hence their performance.

The second section describes our conceptualization of how schedule pressure affects team behavior and consequently project management performance. The third and fourth sections describe, respectively, the research method and the results of the simulations. The article concludes with a discussion of the results and contributions of our study.

Agile Software Development and Schedule Pressure

Agile development approaches appear in many variants (e.g., Scrum, feature-driven development, and extreme programming). Dybå and Dingsøyr (2008) provide an overview of the most referenced agile development approaches. Iterative progress or developing software in iterative cycles is one of the common characteristics of these approaches (Glaiel, Moulton, & Madnick, 2013). An iteration is a time-boxed period of fixed length (Drury, Conboy, & Power, 2012). Iterations are interspersed by customer reviews. Agile software development teams work under high pressure to deliver working software in these short iterations (Drury et al., 2012; Fitzgerald, Hartnett, & Conboy, 2006; Fowler & Highsmith, 2001). Deadlines dictate many aspects of team behavior, such as the pacing of activities (Marks, Mathieu, & Zaccaro, 2001). Because of the iterative nature of agile approaches, teams applying these approaches have to deal with a series of deadlines during the project rather than just one deadline at the end of the project. Because these deadlines are often fixed and negotiated with customers they may cause schedule pressure. Schedule pressure occurs when the time available is inadequate (i.e., the ratio of the number of days required to the number of days available exceeds one (Abdel-Hamid & Madnick, 1991; Austin, 2001; van Oorschot, Sengupta, Akkermans, & Van Wassenhove, 2010). Although schedule

pressure is widely documented in the software and product development literature (e.g., Austin, 2001; Perlow, 1999; Schatz & Abdelshafi, 2005), previous studies demonstrate mixed effects on team behavior and project management performance. Furthermore, little is known about the impact multiple iterations may have on schedule pressure and, consequently, team behavior and project management performance.

Schedule pressure can have positive effects on team behavior and project management performance; it increases team motivation to achieve good results, create knowledge, and demonstrate capabilities (Gardner, 2012). Previous research results indicate that creative individuals and teams may benefit from various constraints, such as schedule pressure (Rosso, 2014), for example, through higher levels of problem solving (Maruping et al., 2015; Pearsall et al., 2009). Schedule pressure may also act as a *pacer*, which allows teams to adjust the task execution rate to the remaining time (Kelly & Karau, 1999; Maruping et al., 2015). In the absence of such pacing, teams may use time by being less productive in the beginning (Buehler, Griffin, & Ross, 2002; Gevers, van Eerde, & Rutte, 2001; Goldratt, 1997; Karau & Kelly, 1992; Latham & Seijts, 1999; Lyneis & Ford, 2007; Seers & Woodruff, 1997), frequently through gold-plating (i.e., refining features beyond requirements) or because of procrastination (Wu, Ramachandran, & Krishnan, 2014). Higher productivity or task execution rates can be realized by working overtime (Abdel-Hamid & Madnick, 1991; Boehm, 1981; Perlow, 1999; Repenning & Serman, 2002). Working overtime increases the output of the team and adds to its collective experience. Experience that increases each day the team works together creates a growing stock of knowledge, which can be applied to improve the team's productivity (Argote, 1999). Thus, working overtime, caused by schedule pressure, enables the team to learn faster (Boh, Slaughter, & Espinosa, 2007; Ford & Serman, 1998; Kessler & Bierly, 2002; Wiersma, 2007).

The many negative effects of schedule pressure have also been reported; for example, Perlow (1999) found that schedule pressure caused developers to use time inefficiently. Furthermore, it may lead to decreased cognitive functioning (Ellis, 2006), reduced creativity (Amabile, 1988), and team members' psychological withdrawal from the task (Gardner, 2012; Pearsall et al., 2009). Beyond a certain point, working overtime can lead to exhaustion, breakdown, or burnout (Moore, 2000). A breakdown occurs when exhaustion levels reach a specific threshold; at that point, team members are unable (or unwilling) to work overtime (Abdel-Hamid & Madnick, 1991; Schaufeli, Bakker, & van Rhenen, 2009). Progress stagnates, but schedule pressure continues to increase. The shorter the time to decompress after a deadline has passed, the more vulnerable the team is to an exhaustion breakdown. Work overload contributes most to exhaustion levels (Abdel-Hamid & Madnick, 1991; Oliva & Serman, 2001; Schatz & Abdelshafi, 2005), and much more so than other factors, including role ambiguity,

conflict, and lack of autonomy or rewards (Moore, 2000). Moore (2000) found that IT professionals experiencing higher levels of exhaustion reported stronger intentions to leave their jobs, a finding consistent with that of other research (Oliva & Stermann, 2001). Karau and Kelly (1992) found that higher levels of schedule pressure had a negative impact on quality and creativity. Schedule pressure leads to an increased rate of errors (Abdel-Hamid & Madnick, 1991; Bjarnason, Wnuk, & Regnell, 2012; Lyneis & Ford, 2007; Wickens, 1984; Zellmer-Bruhn, 2003). Developers under pressure to meet a due date may take shortcuts to deal with complications. Shortcuts are not in the best interest of the project and they usually have adverse effects that lead to extra (re)work later (Austin, 2001; Cao, Ramesh, & Abdel-Hamid, 2010; Oliva & Stermann, 2001). Errors need to be fixed once they are detected, which increases the workload further. A vicious cycle is thus created where high schedule pressure leads to higher error rates; the errors that are detected lead to more work, which increases schedule pressure even more. The errors that remain undetected result in a software product of lower quality (see also Austin, 2001; Thomke, 1998).

As such, schedule pressure can have both positive and negative effects on team behavior and project management performance, which is why some researchers call it a double-edged sword (Gardner, 2012). Table 1 summarizes the positive and negative effects on schedule pressure. Additionally, the existence of a nonlinear relationship between pressure and performance—known as the Yerkes-Dodson law in organizational research—has been recognized (Amabile, Hadley, & Kramer, 2002; Baer & Oldham, 2006; Nan & Harter, 2009; Ohly, Sonnentag, & Pluntke, 2007). This implies that a certain level of schedule pressure exists that causes teams to benefit from its positive effects while preventing its negative effects.

Iteration lengths in agile development projects influence the level of schedule pressure that teams experience. Short iterative cycles prevent optimistic planning and procrastination, because iterative planning helps to divide the overall project into several short-term steps and goals (Conforto, Salum, Amarai, Da Silva, & de Almeida, 2014; Cooper, 1994, 2008). Focusing on smaller parts of a project with intermediate deadlines increases planning accuracy and performance through an increased sense of urgency. As such, progress is better distributed throughout the project, not back-loaded, as are the surprises that result in extra work. Customers provide frequent feedback, thus little effort is wasted on implementing incorrect or obsolete specifications. Early evidence on the effectiveness of agile approaches suggests that they lead to improved accuracy of estimates and lower effort overruns (Molokken-Østvold & Jørgensen, 2005), reduced time to market (Schatz & Abdelshafi, 2005), fewer defects (Lindstrom & Jeffries, 2004), and better quality (Maruping et al., 2009). However, because these short iterations create deadlines that are never distant in time, teams in agile projects may experience peaks of schedule pressure more often than in traditional projects with only one deadline.

Figure 1 presents a stylized view of how sequential (traditional) and agile development influence the behavior of schedule pressure during the project. In sequential development, the team works toward one final due date. On the other hand, in agile development, *each* iteration constitutes its own due date. These due dates often create some pressure: work needs to be finished before this deadline and a customer review needs to be prepared, and so forth. Thus, in agile development there are multiple peaks of schedule pressure, typically occurring toward the end of each iteration. Thus, as Figure 1 shows, agility influences the distribution of schedule pressure during a project's life cycle, which may have important consequences for team behavior and project management performance. This is why it is important to examine how iteration length influences agile projects.

Iterations cannot be treated as “mini-projects,” in the sense that schedule pressure is assumed to be the same for each and every iteration. Although the team may deliver independent software features during each iteration (which resembles a “mini-project”), the software team evolves from one iteration to the next; for example, knowledge gained in the first iteration may be used in the next, thereby increasing team productivity. Team members may quit the team during one iteration, which may negatively impact performance during the next iteration. Feelings of exhaustion that may result at the end of one iteration, may still linger on during the next iteration. Ericksen and Dyer (2004) analyzed team dynamics in projects using only one deadline. They found that events early in the project might set off virtuous or vicious cycles of team performance during the project. As such, it is likely that what happens with the team in early iterations impacts team behavior in later iterations.

Therefore, we have formulated the following research question: How will different iteration lengths (i.e., level of agility, or the number of deadlines) in a project affect schedule pressure, and consequently, how will schedule pressure influence the behavior of the team over time and the project management performance in terms of quality, costs, and time? As such, we examine if there is an optimal length of iterative cycles and, if so, what this optimal length is. This is an important question for both theory and practice, because it addresses the double-edged sword of schedule pressure identified in the literature. In addition, it answers to the call made by Maruping et al. (2015) that it would be more informative for theory to analyze the circumstances under which schedule pressure has a positive versus negative effect on teams. Furthermore, it analyzes the practices that are widely adopted (like the 20-working-day cycle) without solid evidence of their effectiveness.

Method

Model Description

We analyze our research question through a simulation experiment using a system dynamics model of software development

Table 1. Effects of Schedule Pressure**Effects of schedule pressure on team behavior**

Positive effects of schedule pressure on team behavior (learning and productivity)

References:

1. Schedule pressure leads to higher levels of problem solving (Maruping et al., 2015; Pearsall et al., 2009) and increases motivation to achieve good results, create knowledge, and demonstrate capabilities (Gardner, 2012)
2. Productivity of project teams increases because of learning by doing (Ford & Sterman, 1998; Sterman, 2000; Kessler & Bierly, 2002; Boh et al., 2007; Wiersma, 2007)
3. Teams close their mental accounts at the end of an iteration and start the next iteration with a “clean sheet” (Thaler, 1999) As such, team productivity, interrupted by the review, drops at the start of the next iteration before it increases again because of learning by doing (Seshadri & Shapira, 2001)

Type of feedback loop: reinforcing

Endurance of effect: current iteration and the next iterations

Negative effects of schedule pressure on team behavior (exhaustion and turnover)

References:

4. Schedule pressure decreases cognitive functioning (Ellis, 2006), and could lead to members' psychological withdrawal from the task (Pearsall et al., 2009; Gardner, 2012)
5. Long periods of overtime lead to exhaustion (Abdel-Hamid & Madnick, 1991; Moore, 2000; Schatz & Abdelshafi, 2005)
6. Turnover of team members is positively influenced by the exhaustion level (Moore, 2000; Oliva & Sterman, 2001; Schatz & Abdelshafi, 2005)
7. Exhaustion decreases gradually, and only when the team is not working overtime (Abdel-Hamid & Madnick, 1991)
8. Hiring new team members takes time (Sengupta et al., 1999), in addition new team members need to be trained by the experienced ones (Abdel-Hamid & Madnick, 1991; Brooks, 1995; van Oorschot et al., 2010)

Type of feedback loop: reinforcing

Endurance of effect: current iteration and the next iterations

Effects of schedule pressure on project management performance (time, costs, quality)

Positive effects of schedule pressure on project management performance (pacing and overwork)

References:

9. Schedule pressure acts as a “pacer,” which allows teams to adjust the task execution rate to the remaining time (Kelly & Karau, 1999; Maruping et al., 2015)
10. Schedule pressure leads teams to work overtime (Abdel-Hamid & Madnick, 1991; Boehm, 1981; Perlow, 1999; Repenning & Sterman, 2002)
11. When workload is underestimated and the due date is distant, a team will make less progress initially because of inactivity or attention to other matters (Buehler et al., 1994; Gevers et al., 2001; Goldratt, 1997; Karau & Kelly, 1992; Latham & Seijts, 1999; Lyneis & Ford, 2007; Seers & Woodruff, 1997)
12. Short iterative cycles prevent optimistic planning and procrastination, because the overall project is divided into several short-term steps and goals and due dates that are always near (Conforto et al., 2014; Cooper, 1994, 2008)

Type of feedback loop: balancing

Endurance of effect: current iteration only

Negative effects of schedule pressure on project management performance (errors and rework)

References:

13. Schedule pressure reduces the percentage of time spent on quality assurance (Austin, 2001; Oliva & Sterman, 2001)
14. Schedule pressure leads to more errors (Abdel-Hamid & Madnick, 1991; Austin, 2001; Lyneis & Ford, 2007; Oliva & Sterman, 2001; Wickens, 1984; Zellmer-Bruhn, 2003)

Type of feedback loop: reinforcing

Endurance of effect: current iteration only

(developed in iThink software). System dynamics is especially suitable for situations involving multiple and interacting processes, time delays, and other nonlinear effects such as feedback loops (Davis, Eisenhardt, & Bingman, 2007). This modeling approach is used in similar research (Abdel-Hamid, 2010; Akkermans & van Oorschot, 2016; Cao et al., 2010; Dutta, 2001; Lyneis & Ford 2007; Oliva & Sterman, 2001; Repenning, Goncalves, & Black, 2001; Repenning & Sterman, 2002).

We use an extensively validated model of software development, summarized in Figure 2. The model is a rich and widely accepted representation of the dynamics of software projects, and has been used in the research on software development (e.g., Abdel-Hamid, 2010; Abdel-Hamid, Sengupta, & Swett, 1999; Sengupta & Abdel-Hamid, 1993; Sengupta & Abdel-Hamid, 1996; Sengupta, Abdel-Hamid, & Van Wassenhove, 2008). The model integrates the

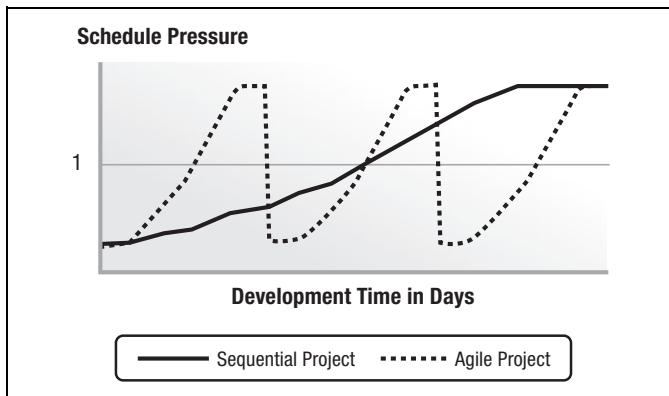


Figure 1. Schedule pressure in sequential and agile projects.

multiple functions of the software development process, including management functions (e.g., planning, controlling, staffing) and software production (e.g., designing, coding, reviewing, testing). The original model, developed by Abdel-Hamid and Madnick (1991), based on the field interviews of software project managers in five organizations, was complemented by an extensive database of empirical findings from the literature. Abdel-Hamid and Madnick used the following sets of tests to validate the model: First, a face validity test was performed to test the fit between the rate/level/feedback structure of the model and the essential characteristics of real project environments; this fit was confirmed by the software project managers involved in the study. Second, reference modes were replicated to test whether the model can endogenously reproduce the various reference behavior modes characterizing real environments. Finally, five case studies were conducted to test the model. The objective of these case studies was to examine the model's ability to reproduce the dynamic patterns of a completed software project (Abdel-Hamid & Madnick, 1991). Two case studies were reported in work by Abdel-Hamid and Madnick (1991) and Abdel-Hamid (1993); the other case studies were conducted independently at Bellcore (Glickman & Kopcho, 1995), Mitre (Powell, 1987), and Bell Laboratories (not published). All case studies were conducted in organizations other than the five organizations studied during model development (see Abdel-Hamid & Madnick, 1991, for a full description of the structure and formulation of their original model). In the Appendix at the end of the article, we present a full description of our model, which is adapted to simulate multiple iterations. The simulation model is available from the authors upon request. Following, we describe only the main variables and relationships in our model.

Figure 2 shows a stylized model of software development. The core process of software development is shown in the two boxes at the top of Figure 2. Software development is conceptualized as transforming a stock of tasks that need to be developed into a stock of developed tasks. A *task* is a set of deliverables, such as lines of code or source instructions. Initially, most of the tasks that need to be developed are known

and are in the stock of *Tasks to be Developed*. The stock *Tasks Developed* is initially empty because nothing has been developed yet; once completed, a task flows from *Tasks to be Developed* to *Tasks Developed*. The *software development rate* is the speed of this task flow. The rate is determined by the team's productivity, that is, the number of tasks developed in a normal working day. Higher productivity leads to more tasks flowing to *Tasks Developed* and the project is completed sooner. The team's workload increases with the discovery of new tasks while executing known tasks (Bjarnason et al., 2012; Dawson & Dawson, 1998; Eisenhardt & Tabrizi, 1995; Iansiti & McCormack, 1997; McDermott, 1999; Tatkonda & Rosenthal, 2000; van Oorschot, Bertrand, & Rutte, 2005). This is represented by a causal link between *software development rate* and *discovery rate of new tasks* in Figure 2. When new tasks are discovered the level of *Tasks to be Developed* increases; as such, these additional tasks increase workload and, consequently, schedule pressure. The project is complete when there are no tasks left in *Tasks to be Developed*, and all tasks are in *Tasks Developed*. To accommodate for agile development, the structure and process described above and depicted by the two boxes (stocks) at the top of Figure 2 are repeated for each iteration. This simply means that *Tasks to be Developed* is filled and *Tasks Developed* is depleted at the start of each iteration.

The process of developing tasks as described earlier is consistent regardless of iteration length. In both situations, whether the project is done using an agile or sequential approach, tasks need to be developed and these tasks will flow from one stock to the next. The difference is that for projects with long iterative cycles, the number of tasks that need to be developed in that cycle will typically be higher and the deadline will be more distant. To examine how iteration length (i.e., the number of pressure peaks) influences team behavior and project management performance, we need to consider both the positive and negative effects of schedule pressure.

The positive effects of schedule pressure are captured by the balancing *overwork loop* (depicted by the dashed arrows in Figure 2) and the reinforcing *learning loop* (depicted by the long dashed arrows in Figure 2). When schedule pressure increases, the team realizes it is running out of time. To compensate for this, the team increases its development rate through overtime. This process continues until all tasks are developed or schedule pressure decreases again. When no new tasks are discovered, overtime has a balancing effect on schedule pressure: a higher development rate reduces the number of tasks remaining. A higher development rate also has a positive effect on the cumulative experience of the team. A team's level of experience determines its productivity via the learning curve (Boh et al., 2007; Ford & Sterman, 1998; Kessler & Bierly, 2002; Wiersma, 2007). The higher the productivity, the higher the development rate, which only reinforces the cumulative experience. As such, experience and productivity are higher at the end of an iteration than at the beginning.

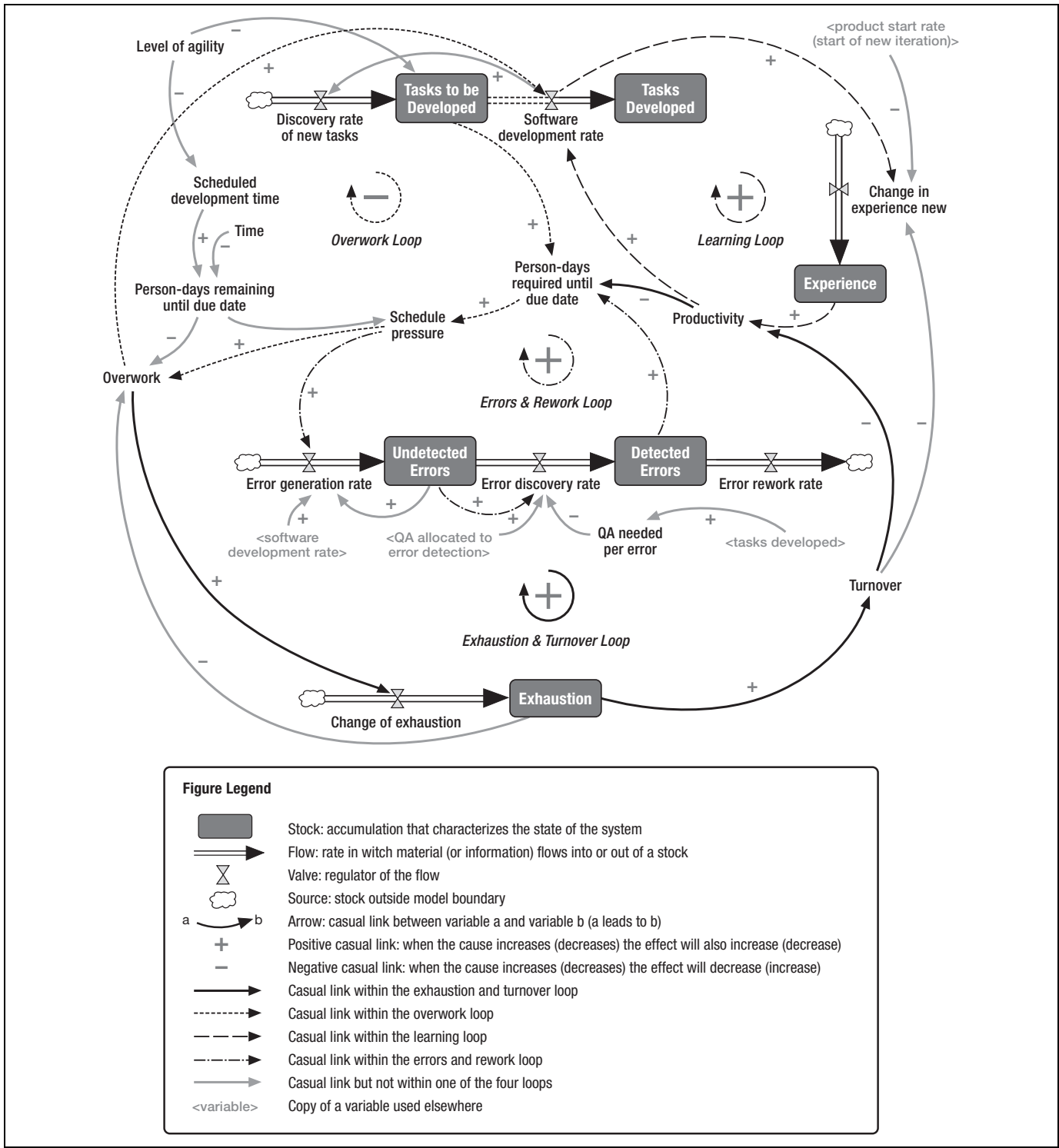


Figure 2. Model overview.

The negative effects of schedule pressure are described by two reinforcing loops: *exhaustion and turnover* (depicted by the solid black arrows in Figure 2) and *errors and rework* (depicted by the long dashed dotted arrows in Figure 2). Exhaustion is modeled as a stock that increases

whenever the team works overtime to complete an iteration. Exhaustion gradually decreases in the beginning of the next iteration as long as the team does not feel the need to work overtime. As soon as schedule pressure increases in this next iteration, the level of exhaustion increases when the

team resumes working overtime again. Beyond a point, working overtime can lead to exhaustion, breakdown, or burnout. Progress stagnates, but schedule pressure continues to increase. Furthermore, exhaustion may trigger attrition; in Figure 2, this is depicted by the turnover rate. Turnover reduces team size immediately, thus leading to a drop in team productivity (i.e., fewer tasks completed per day by the team). New employees hired as replacements typically have less experience and lower productivity than former employees, which is why exhaustion and turnover are connected with schedule pressure through a vicious cycle (reinforcing loop): An increase in schedule pressure leads to increased exhaustion and turnover that reduce productivity and, as a result, increase schedule pressure even more.

Schedule pressure also increases the rate of errors. Once detected errors need to be fixed and this further increases the workload. A vicious cycle is thus created wherein high schedule pressure leads to higher error rates; the errors that are detected lead to more work, which increases schedule pressure even more, which is why the errors and rework loop is reinforcing. The errors that remain undetected at the end of the project result in a lower-quality software product.

In this subsection, we described four feedback loops that describe the effects of schedule pressure on team behavior and hence, project management performance. The feedback loops can be balancing or reinforcing, and their impact on performance can be good or bad. As stated earlier, what happens in one iteration can spill over to the next (Erickson & Dyer, 2004), which is why we cannot treat iterations as independent mini-projects. Potential spillovers in our model are learning, and exhaustion and turnover. Although team members may be experienced at the start of the first iteration, it is likely that they are relatively inexperienced with the customer, the requirements, or with each other. Team productivity, therefore, tends to be lower in the beginning of the project and gradually increases as more and more tasks are developed via the learning curve. Iterations may disrupt the continuity of learning. At the end of an iteration, the team must stop working on developing tasks in order to prepare for the customer review. After this review, it takes time for the team to “close their mental accounts” (Thaler, 1999) and decompress before preparing for the next iteration; consequently, productivity suffers because of this iterative way of working (Seshadri & Shapira, 2001). Thus, productivity at the *start* of iteration $i + 1$ is lower than at the end of iteration i . However, because the drop in productivity is less than the productivity gained during the previous iteration, productivity at the *end* of iteration $i + 1$ still is higher than at the end of iteration i . In other words, average productivity increases during the project despite these disruptions.

There is a positive spillover effect from tasks done in early iterations to team productivity in later iterations. Exhaustion and turnover may induce a negative spillover effect. At high agility levels, due dates follow each other rapidly. Thus, the

time available to decompress after working overtime in the previous iteration may be insufficient to empty the “exhaustion stock.” Consequently, exhaustion builds up from one iteration to the next, and may eventually cause a breakdown. In the worst-case case scenario, exhaustion leads to increased turnover. Because of hiring and training delays, team productivity will be lower in iterations that immediately follow any turnover.

Independent and Dependent Variables

The target of our simulations is a real-life, medium-sized organic project developed in-house for satellite telemetry software. The project consists of 24,400 Delivered Source Instructions (DSI); however, because the full size of the project is not clear to the team at the beginning of the project, the perceived project size is smaller than the actual project size. We assumed that 35% of all required tasks are actually unknown at the start of the project; this figure is the average of estimates obtained from interviews with managers of agile projects and conforms to findings in the literature (e.g., Abdel-Hamid & Madnick, 1991; Repenning et al., 2001; van Oorschot et al., 2005). This uncertainty fraction of 35% leads to an underestimation of workload and, consequently, it will create schedule pressure (Eisenhardt & Tabrizi, 1995; Iansiti & McCormack, 1997; Dawson & Dawson, 1998; McDermott, 1999; Tatikonda & Rosenthal, 2000). We use COCOMO equations (Abdel-Hamid & Madnick, 1991; Boehm, 1981) to estimate the perceived project size in person-days from the number of DSI that the project should deliver. Project managers make plans based on their perceptions of project size (and, therefore, workload). Thus, according to COCOMO: the *planned person-days for project* (945 person-days) = $2.73 \times 19 \times (\text{perceived project size} \div 1,000)^{1.05}$. The *perceived project size* is equal to the *actual project size* that is corrected for uncertainty (*perceived project size* (15,860 DSI) = *actual project size* (24,400 DSI) $\times$ (1-uncertainty fraction)). The project should be completed in 260 days (which is comparable with the target development time of the project simulated by Abdel-Hamid and Madnick, 1991). Team size is set to 4.9 people. With this size, the team perceives it has sufficient time for the project ($945/4.9 < 260$); however, this size turns out to be too small when all tasks are known. A project of 24,400 DSI requires at least 1,485 person-days, and a team of 4.9 people requires a development schedule of 303 days; hence schedule pressure increases, requiring overwork. The scheduled duration of the project is 260 working days when executed in a purely sequential mode. At the other extreme, the highest level of agility we examine has a monthly iteration length (i.e., 20 working days). From this perspective, sequential development can be viewed as an extreme case, wherein the project is implemented in a single iteration (Agile-1). In contrast, a truly agile project is divided into several iterations (Agile- n). In our model, Agile-13 is the highest level of agility. Our simulation consists of multiple runs of the same

Table 2. Team Behavior and Project Management Performance

Level of agility	Iteration length (in days)	Schedule Pressure		Team Behavior					Project Management Performance		
		Average schedule pressure during project	Total number of days with schedule pressure above 1	Average productivity (DSI per person per day)	Total number of days with overtime	Total number of people that quit the team	Total number of person-days allocated to QA	Total number of errors reworked	Quality (total number of undetected errors)	Costs (total number of expended person-days)	Time (duration of project, total number of days)
1	260	1.16	143	17.92	62	1.97	240	450	239	1361	280
2	130	0.98	119	18.90	73	1.82	238	462	198	1291	262
3	87	0.91	105	18.97	74	1.82	240	468	179	1286	261
4	65	0.90	100	18.99	74	1.85	239	471	164	1285	261
5	52	0.90	97	18.90	74	1.83	239	473	160	1291	262
6	43	0.91	95	18.89	75	1.83	237	470	164	1292	262
7	37	0.95	96	18.87	77	1.84	235	466	174	1293	262
9	33	1.02	105	18.61	89	1.96	230	469	189	1311	264
11	24	1.24	140	18.68	88	1.95	214	443	272	1306	263
13	20	1.50	183	18.74	82	1.97	194	409	416	1302	264

project, each with a different level of agility (i.e., iteration length). This approach allows us to examine how the level of agility affects team behavior and project management performance. Following the literature, we assume that all iterations at a particular level of agility have the same length (Larman, 2006; Lindstrom & Jeffries, 2004; Schatz & Abdelshafi, 2005; Vigden & Wang, 2009).

Schedule pressure is our main dependent variable, which we analyze in three different ways. First, *average schedule pressure* for a period is calculated as the sum of daily schedule pressure during the period divided by the number of days in that period. Second, the *behavior over time of daily schedule pressure* is analyzed during the project in terms of the pattern of oscillation. Finally, the *cumulative number of days of schedule pressure* (total number of days on which schedule pressure exceeded 1) is examined to determine for how many days of the project the team felt pressured and when this pressure occurred.

Team behavior is measured using the following five variables. First, *average productivity (DSI per person per day)* is the ratio of the total number of DSI of the project (which is the same number for all levels of agility) and the total number of person-days expended (including overtime). Second, the *total number of person-days in overtime* measures the total number of person-days spent on top of normal working hours, meaning working overtime. Third, working overtime for too long leads to exhaustion, which increases the quit rate. The *total number of people that quit the team* is the accumulation of this quit rate, that is: the sum of all people who have quit the team during the project. Fourth, *total number of person-days allocated to QA* describes the amount of capacity allocated to quality assurance (detection of errors) in the project. Finally, and fifth, the *total number of errors reworked* measures how many errors were detected and reworked during the entire project. Note, that when

quality assurance (QA) is reduced not all errors are detected.

In addition to schedule pressure and team behavior, project management performance is also measured in terms of quality, costs, and time. The principal effect of schedule pressure on agility results from creating peaks of workload, which primarily affect quality. Thus, the dependent variable reflecting quality is the *number of undetected errors* remaining. A software product with many undetected errors (bugs) is more inferior in quality than one with fewer undetected errors. Errors detected require effort to fix, thus affecting two additional dependent variables widely used in projects: the *number of person-days expended* or *costs* (including any overtime expended) and *development time* (number of working days needed to complete the project).

Results

Project Management Performance

Table 2 displays the results on schedule pressure (average and cumulative), team behavior (productivity, overwork, quit rate, quality assurance, and errors reworked), and project management performance (quality, costs, and time). Most variables (except for quality assurance) show improvements with increasing levels of agility. Yet, in general, the improvement sustains only up to the Agile-4 to Agile-6 level and reverses at higher levels. In other words, there is a U-shaped trend; to better understand the reasons underlying this U-shaped trend, we examine how the level of agility drives schedule pressure and thus affects quality, costs, and time. (Please note that following the finding that project size is a boundary condition for agile development (Harris, Collins, & Hevner, 2009), we also conducted experiments with a project half the size (i.e., 12,200 DSI); the results are similar to those reported here.)

Schedule Pressure and Performance

Figure 3 plots average schedule pressure during the project for each level of agility. Figure 4 shows the patterns of schedule pressure over time for three levels of agility: Agile-1, Agile-6, and Agile-13. Figure 5 shows the aggregate impact of schedule pressure over time, in other words, the *cumulative* number of days on which schedule pressure exceeds 1. (To facilitate the ease of visual exposition, the other levels of agility are not shown in Figures 4 and 5.)

The average schedule pressure across various levels of agility (see Figure 3) exhibits a distribution similar to that of the dependent variables. The edges of the distribution and the large, “flat” area in between suggest the existence of a range of agility levels in which average schedule pressure is moderate. This range encompasses levels Agile-3 through Agile-6. Beyond this range, schedule pressure shows an upward drift. As shown in Table 2, higher schedule pressure seems to negatively impact project management performance. We now explore how the project team’s behavior is affected by the downward and subsequent upward slopes of schedule pressure arising from increased levels of agility.

Increase in Agility from Level 1 (260 days’ iteration) to Level 6 (43 days’ iteration)

For each agility level, schedule pressure is low at the start of an iteration and then peaks at the end (see Figure 4). However, the heights and durations of the peaks differ both within and across different agility levels. For Agile-1, it takes 137 days (more than 50% of the scheduled development time) before the team realizes that the time remaining is less than the time required. Schedule pressure never drops after that day and instead accumulates rapidly (see Figure 5). In these first 137 days, the team has underestimated total development effort and has underutilized time. By the time the team recognizes the need to work much harder, it is likely too late. Thus, the sequential plan-based approaches embodied in Agile-1 are typically characterized by low schedule pressure for a significant part of the project, followed by a steep rise. Figure 4 shows that the rise in schedule pressure for the Agile-1 project is even higher after day 211. This is when the team has a breakdown due to exhaustion after working overtime for too long.

Now consider agile development, when the due date for the first deliverable arrives earlier than in the case of Agile-1—for example, in Agile-6 (Agile-6 is the example chosen here because this level of agility lies between the two extremes of Agile-1 and Agile-13; moreover, Agile-6 has much better project management performance than either Agile-1 or Agile-13; see Table 2). In order to complete the iteration on time, the team must make progress from the beginning. Thus, schedule pressure arrives earlier: it takes the team only 33 days to notice that the time required for completing the iteration is less than the time remaining (see Figure 4). In addition and as shown in Figure 5, the manner in which schedule

pressure recurs over the project is also different. The trajectory of the cumulative line for Agile-6 is flatter than that for Agile-1 (the respective slopes for Agile-1 and Agile-6 are significantly different: $F = 12,857.39$, $p < 0.01$); in other words, schedule pressure is more spread out over the life of the project. Thus, up to Agile-6, the effect of increasing agility on schedule pressure is that pressure is experienced earlier, but has less of a cumulative impact. This explains the downward (followed by essentially the flat) path of average schedule pressure for levels Agile-1 through Agile-6 as shown in Figure 3. Therefore, from sequential development until Agile-6, we find that with increasing levels of agility, progressively less time is wasted in the beginning and more work is accomplished earlier. Further, as the lower schedule pressure peaks at the end of the Agile-6 project, as shown in Figure 4, less work needs to be done at the end. The result is an increasing cycle of learning and productivity. Earlier and higher schedule pressure triggers overwork: Overwork is 8.5 days for iteration 1 of Agile-6 versus none for the corresponding period in Agile-1; the results on team behavior presented in Table 2 corroborate this. (Please note that each simulation we ran gives us detailed performance results for each day in the project [260 days]. All these results are recorded in an Excel file and available from the authors upon request. Table 2 presents only the overall results for the entire project, not per iteration.) Increasing the level of agility up to Agile-6 leads to more overtime, but not to such an extent that it leads to increased exhaustion and increased turnover. Overwork leads to more learning, and in turn, increased productivity (*overtime and learning* loops in Figure 2): average productivity in iteration 1 for the Agile-6 level is 18.48 DSI/person-day, versus 14.36 DSI/person-day for the corresponding period in Agile-1. The average productivity over the entire project is also higher for Agile-6 than for Agile-1. Because of higher productivity, more work gets done early; moreover, the level of work remaining becomes evident early—in other words, there is earlier and better visibility regarding task uncertainty and the rework required. This visibility helps the team to manage workload better in the latter parts of the project. The team is already productive, and good progress ensures that there is less continuing schedule pressure and overwork, which reduces the team’s inclination to take shortcuts that feed the *errors and rework loop* (see Figure 2). Furthermore, more time is available for quality assurance, which leads to a higher number of errors that are reworked (and consequently a higher quality).

The results are displayed in contrasting performance in the first and last 90 days of the project. In the *first* 90 days, the Agile-1 team develops 5,517 DSI at a productivity of 13 DSI/person-day and defect density of 27 errors/KDSI. In contrast, the Agile-6 team develops 8,133 DSI (47.4% more than Agile-1) at a higher productivity rate of 18 DSI/person-day and a lower defect density of 23/KDSI. In the *last* 90 days in this scenario, the Agile-1 team catches up on early inefficiencies to develop 13,446 DSI at a higher productivity of 24 DSI/person-

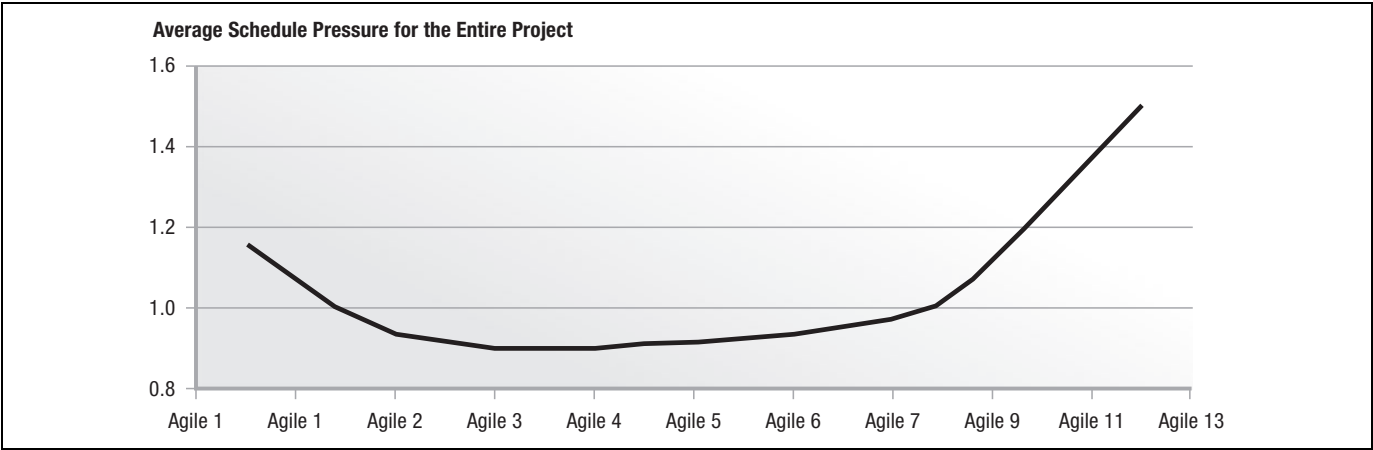


Figure 3. Average schedule pressure.

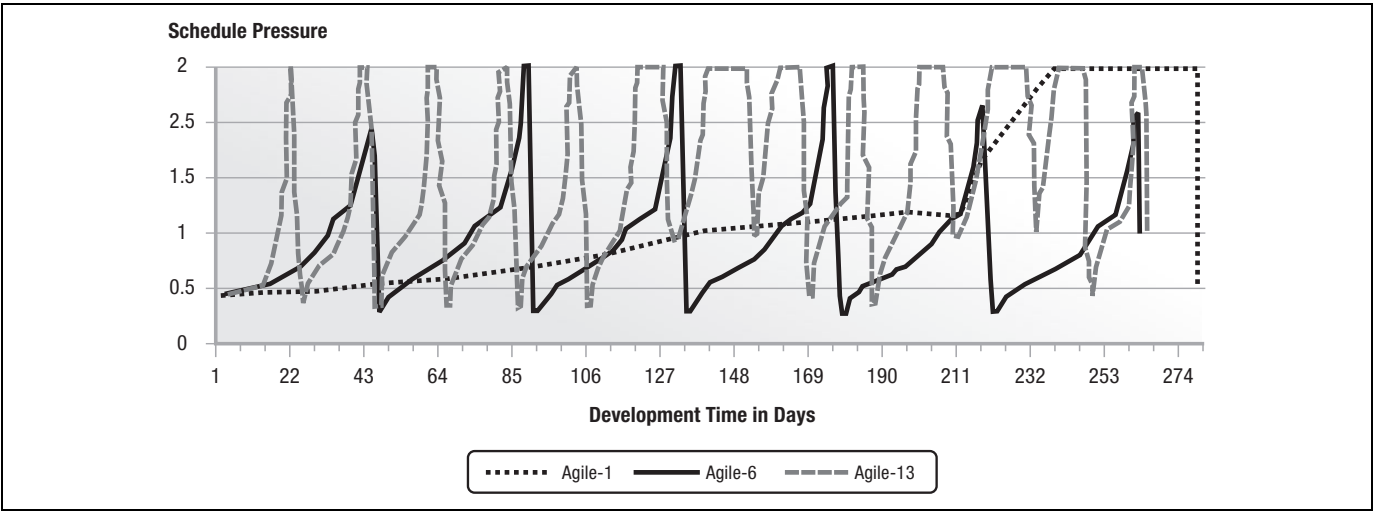


Figure 4. Schedule pressure during the project.

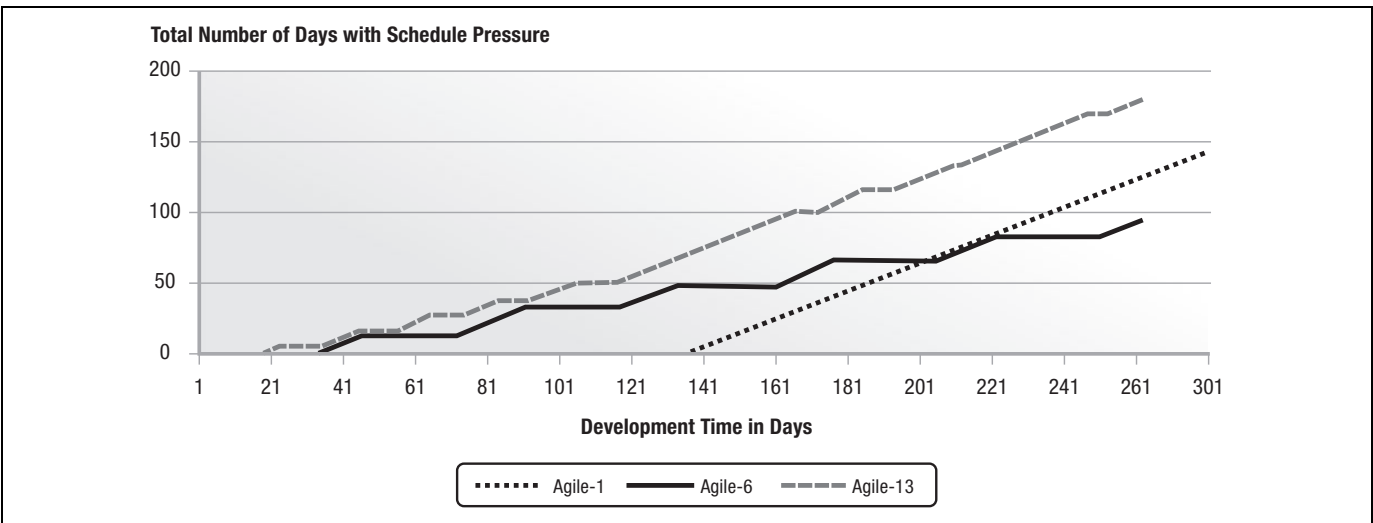


Figure 5. Total number of days with schedule pressure.

day—but with a considerably higher defect density of 32/KDSI, triggered by massive and unsustainable schedule pressure. In comparison, the Agile-6 team needs to develop only 8,916 DSI, with steady productivity (20 DSI/person-day) and a lower defect density (24 errors/KDSI).

Long iteration lengths (e.g., in plan-based approaches such as sequential development) create little schedule pressure initially, leading to *inefficiency caused by inactivity*. Schedule pressure eventually increases to unsustainable levels as the team discovers that it has not made enough progress. Reductions in iteration length bring the next delivery date progressively closer, thus creating increased schedule pressure. Higher schedule pressure leads to more work being done, which, in turn, accelerates the learning curve and improves productivity, thus kicking off a virtuous cycle that strengthens as the level of agility increases from Agile-1 to Agile-6.

Increase in Agility from Level 6 (43 days' iteration) to Level 13 (20 days' iteration)

Higher levels of agility with earlier due dates require even earlier progress. However, progress is not linear: productivity is low in the beginning, and the learning curve takes time to kick in. Therefore, even though the amount of work required for the first deliverable is lower at higher levels of agility (7.7% of the total project for Agile-13 versus 16.7% for Agile-6), the time available is also proportionately lower. Consequently, teams at higher levels of agility must spend more effort per DSI to get the work done initially; they also face greater schedule pressure earlier in the project. For example, as shown in Figure 5, schedule pressure for Agile-13 starts much earlier (day 18) than for the other levels shown. Moreover, pressure also accumulates to increasingly higher levels. The slope of cumulative pressure for Agile-13 is significantly different from that of Agile-6 ($F = 3,587.39$, $p < 0.01$). The Agile-13 project counts 183 days with schedule pressure above 1 versus 95 days for the Agile-6 project (see Table 2).

Thus, beyond a point (Agile-6), it becomes counterproductive to create increasingly earlier deadlines for the first few deliverables because it takes time for the learning effect to occur. The productivity of the team is low in the beginning, and more effort is required to get the job done. For example, delivery of the first iteration requires 3.3 days of overwork for the Agile-13 team versus none for the Agile-6 team during the same period. Since time is required for learning, however, the higher level of overwork does not result in any productivity gain for Agile-13 over Agile-6. Indeed, in the first 90 days, the Agile-13 team actually develops slightly fewer DSI than the Agile-6 team (8,100 versus 8,133 DSI) and with poorer quality (defect density of 27 versus 23 errors per KDSI). On average, over the entire project (see Table 2) overwork increases while productivity decreases.

The first iteration in the Agile-13 project was completed one day late. Because iteration lengths are equal, the lateness of the

first iteration reduces the time available for the second iteration. This lateness means that the second iteration is almost as difficult for the team to finish on time, even though learning and productivity have increased somewhat by then. Given the need to catch up on the lateness of previous iterations, teams in highly agile projects also experience high peaks of schedule pressure at the end of the project (see Figure 4); thus, higher levels of agility are likely to create delays in subsequent iterations, resulting in late projects.

The relatively poor performance at higher-agility levels is therefore caused mainly by *inefficiency due to overactivity*. The initial review dates are simply too early, and problems from the first iteration cascade to the next. For the Agile-6 level, each iteration is easier to complete than the previous one. However, when agility increases to the level of Agile-13, each iteration is almost as difficult to complete as the previous one. The team is constantly being forced to make more progress than they can handle for their given productivity, often requiring overtime (overactivity) to make up for the low productivity and lateness of previous iterations. More overtime leads to increased levels of exhaustion and turnover, which increase schedule pressure even further. This results in more errors, and lower capacity is available for quality assurance, such that a large number of these errors remain undetected, thereby reducing quality.

Discussion

Agile approaches—such as Scrum, feature-driven development, and lean development—have increasingly gained interest in the software community (Cardozo et al., 2010; Dybå & Dingsøyr, 2008). These approaches are agile because they employ short, incremental, iterative, time-boxed development cycles (Lee & Xia, 2010), thereby enabling teams to deal with changing customer requirements. Despite their popularity, little research has examined their key concepts, and underlying theoretical relationships or a “theoretical glue” to integrate these concepts is lacking (Ågerfalk, Fitzgerald, & Slaughter, 2009; Conboy, 2009; Dingsøyr, Nerur, Balijepally, & Moe, 2012; Fruhling & de Vreede, 2006; Lee & Xia, 2010). As a result, there is not enough understanding of the mechanisms through which agile approaches affect employee attitudes, team behavior, and therefore project management performance (Maruping et al., 2009; Ply et al., 2012).

Our study examines one such mechanism: the level of agility, represented by the length of iterations. Iterations typically have deadlines or end dates at which a customer review is scheduled. As such, iterations in agile projects create a series of deadlines rather than just one deadline at the end of the project. In turn, these deadlines cause schedule pressure to oscillate between peaks that build up to each customer review, and troughs, which occur immediately after the review and at the start of the next iteration. Research on the effects of schedule pressure on team behavior paints a complex picture reporting positive, negative,

and nonlinear relationships (Baer & Oldham, 2006; Gardner, 2012; Maruping, 2015; Nan & Harter, 2009). These studies have in common that they focus on traditional projects or tasks, with only one deadline, not the series of deadlines that agile approaches employ. From a theoretical perspective, little is known about how long these iterative cycles in agile projects should be in order to have a positive effect on team behavior and project management performance.

Through simulations of software projects with varying iteration lengths, we examine the impact of these different iteration lengths. The simulation results show that at one end of the agility continuum (i.e., strictly sequential projects), the eventual peak of schedule pressure is so high (and lasts so long) that it leads to sharp increases in the errors generated, with not enough time to detect and fix them. Due to the distant deadline, the team lacks a sense of urgency in the beginning of the project, which leads to inactivity or procrastination. The result is not only poor quality, but also extra effort in fixing the errors that *do* get detected. At the other end of the continuum (i.e., high levels of agility), deadlines toward the end of each iterative cycle create high peaks of schedule pressure. These occur frequently throughout the project, again with an outcome of poor quality. Thus, it is possible to push a team over the edge by planning so many intermediate deadlines that there is little or no time to recover from the pressure of the previous iteration. Therefore, beyond a certain point (iteration lengths of 43 to 65 working days in our simulations), increasing the level of agility simply adds to schedule pressure without commensurate increases in learning or productivity. Progress is demanded too quickly, whereas an understanding of the task at hand and the resulting productivity are still low (Ramesh, Mohan, & Cao, 2012), which leads to overwork (overactivity). The team falls increasingly behind in deliverables, and exhaustion from high schedule pressure reduces productivity further. Mounting schedule pressure leads to less attention paid to error detection and correction, as well as a greater inclination to take shortcuts that further increase error levels, which result in adverse outcomes, poor quality, and higher effort and time taken. There is a boundary beyond which increased agility may be counterproductive; therefore, shortening iteration lengths any further is unlikely to improve performance and may even decrease it.

Contributions and Limitations

Theoretical Contributions

Iterative cycles used in agile software development not only divide the total amount of work into smaller parts, these cycles also create a series of deadlines for project teams. At each deadline a small part of the project work needs to be finished. As a result, these deadlines can lead to schedule pressure. Schedule pressure can be a double-

edged sword for project teams. Previous literature reports on both the positive (Gardner, 2012; Pearsall et al., 2009; Rosso, 2014) and negative (Austin, 2001; Moore, 2000; Oliva & Sterman, 2001) effects on team behavior and project management performance. Because iteration lengths determine the number of deadlines teams face during a project, iteration lengths will have an impact on the schedule pressure teams perceive during a project. This is why we examined through simulation if there is an optimal iteration length for agile projects and, if so, what this length is. Our simulation results indicate that contrary to widely advocated practice, iterations of 20 working days are less beneficial to project management performance. Iteration lengths that are closer to 43 to 65 working days (two to three calendar months) lead to better performance in terms of time, quality, and costs. Our results build on those of recent studies that opt for a more nuanced perspective on the use of short and fixed iterations and recommend hybrid or balanced practices between traditional and agile development, thus trying to use the best of both worlds (Port & Bui, 2009; Ramesh et al., 2012), flexible iteration lengths (Middleton & Joyce, 2012), and longer iterations (Petersen & Wohlin, 2010).

Software development environments are examples of complex, adaptive systems (Vigden & Wang, 2009), characterized by lack of visibility, delays, and reactivity (Abdel-Hamid et al., 1999). Functioning in such systems requires managers to balance stability and instability (Ramesh et al., 2012; Vigden & Wang, 2009), so as not to tip them into chaos (e.g., situations of high agility in our study). However, as research on dynamic decision environments has repeatedly shown (e.g., Cronin, Gonzalez, & Sterman, 2009), it is not easy to develop the intuition required for managing in such systems; furthermore, such intuition cannot be developed through experience alone (Sengupta et al., 2008). It is instructive to note that software development environments are rife with practices adopted without solid evidence for their effectiveness (Austin & Devin, 2009; Zerkowitz et al., 2003). Their adoption without convincing evidence of effectiveness raises the question of whether the practices are actually useful as now implemented—for example, with a 20-working-day cycle (Conboy, 2009). Meanwhile, the newly instituted iterative practices can create friction in the organization (Ply et al., 2012; Sarker & Sarker, 2009). It is thus essential for research to offer conceptually based guidance on how to consider the impact and trade-offs generated by agile practices, such as working in iterative cycles (Port & Bui, 2009). Because the practices affect the behavior of the system as a whole, the guidance can be useful if based on a systemic understanding of their impact on the key causal relationships (e.g., among overwork, exhaustion and turnover, and errors). Whereas many previous studies offer a piecemeal view on agile development (Conboy, 2009), our study offers a theory-based explanation through a model that combines the technical aspects of software development (e.g., task execution and iteration lengths), with temporal aspects (multiple

deadlines), and with human aspects (team behavior). As such, we answer Ancona, Okhuysen, and Perlow's (2001) call for research to examine the link between temporal context and team functioning. Likewise, we respond to the suggestion of Maruping et al. (2015) to focus on explaining the circumstances under which schedule pressure has a negative versus positive influence on teams. Our findings also contribute to the literature on schedule pressure and show a clear U-shaped relationship between the frequency at which deadlines follow each other (and thus the behavior of schedule pressure during the project) and project management performance. Although U-shaped relationships between schedule pressure and performance have been demonstrated before (Amabile et al., 2002; Baer & Oldham, 2006), this was not done in projects with a series of deadlines that follow each other as frequent as in most agile development projects.

Managerial Contributions

Our results show that the best iteration lengths are closer to 43 to 65 working days (two to three calendar months) than to the 20-working-day monthly cycle typically used in the field. Contrary to widespread practice, there is no "one-size-fits-all" length of iterations. When deciding on the length of an iteration, project managers need to weigh the importance of factors (e.g., volatility of requirements) that drive the search for agility, against potentially significant downsides such as diminished software quality. There should be some alignment between the volatility of requirements generation and the interval at which reviews occur, which is a concept analogous to the co-evolution principle advocated by Vigden and Wang (2009). In development environments, in which specifications are highly volatile, requirements can be specified only for short intervals (e.g., one month or even less). In these cases, iteration lengths of 43 to 65 days may not be appropriate. Such projects require greater flexibility, and the project manager might do better with monthly intervals even at the cost of higher delivery risk (Harris et al., 2009) or reduced product quality.

This does not imply, however, that *all* development situations require such high agility levels. Arguably, shorter iterations can foster a lack of discipline in requirements' generation. Consider the example of organizations with high levels of development capability (such as CMM level 5). Such organizations exercise process maturity by finding ways to *limit* the number of modifications that customers make, typically by working with customers to help them prioritize the pool of potential requirements/changes (Agrawal & Chari, 2007). Our results may be viewed as a temporal version of this argument: moderate levels of agility can improve project management performance by providing flexibility for modifications when needed, but also creating discipline by calibrating the frequency with which they can be specified. Our article provides guidance on how the length of an iteration can influence team behavior and project management

performance. Thus, rather than being driven by fixed notions of monthly iterations, project managers can be better off choosing longer cycles.

Limitations

The results presented in this article should be interpreted in the context of our approach. The model structure and equations are grounded in the literature, and parameter values are based on empirical data from prior studies. Therefore, the evidence level of our model is at least medium, according to the classification scheme for system dynamics models suggested by Homer (2014). A medium level of evidence means that we have good support for the input (the structure) of our model, but not for the output (in terms of behavior). Although our main finding that monthly iterative cycles may be too short for agile projects—and that cycles of 2 to 3 months could lead to better performance—have been acknowledged in the informal conversations we have had with experienced project managers, we still have to validate our findings in a future study. This would bring the level of evidence of our model to the highest level (Homer, 2014). The Method section in this article explains why we employed simulation to explore the research question. A review of the organizational behavior literature has found remarkable similarities between research findings obtained in simulations and real-life situations (Davis et al., 2007). Simulation-based experimentation is particularly advantageous in the study of complex dynamic decision-making environments, such as software projects. Simulations can effectively capture the inter-temporal aspects of tasks, flow of information, and outcomes that are characteristic of such work settings.

Our treatment of changes in customer requirements implicitly assumes that customers change previously defined functionality but do *not* add new functionality over time. This is a limitation, since requirement changes in projects often entail the addition of new features and/or functionality (Maruping et al., 2009). Because adding new functionality can only aggravate the problem of schedule pressure at higher levels of agility, our results can be considered a baseline for comparing the more general case where customers add new requirements. That is, adding new functionality is likely to have the effect of a further decrease in quality as well as higher costs and development time. Likewise, in projects with higher levels of uncertainty (note that we have assumed that 35% of all tasks that need to be executed were unknown in the beginning of the project), schedule pressure is likely to increase when these unknown tasks are finally discovered. This makes the iteration length decision even more important.

Research on agile approaches has also examined the impact of various innovative practices, such as pair programming (Cao et al., 2010) and extreme programming (Fruhling & de Vreede, 2006). The manner in which such practices interact with a project's agility level is an open question that can be investigated successfully in future research.

Conclusions

The primary contribution of this study is to provide a basis for managers to consider whether or how to implement agile development in their organizations. Given the results reported here, organizations can formulate project guidelines based on a clear understanding of the costs and benefits of varying levels of agility. The starting point is to determine whether the requirements are so volatile that they can only be formulated in small increments, and that frequent customer feedback is needed before proceeding to the next step. If so, then using short iterations is a logical approach—perhaps as short as 20 working days but more generally commensurate with the rate at which requirements can be meaningfully discovered. The length of the iterative cycle should not only be aligned with the evolution of customer requirements, the evolution of team productivity (learning curve) should also be taken into account. How easy is it to predict the time required to develop new

requirements? How fast can the team reach full productivity? A higher level of agility can in this case yield benefits compared to traditional development, not only because of frequent customer feedback but also because shorter iterations create a series of deadlines that increase a sense of urgency, pacing of work, and team productivity. Our study also shows that it is possible to push a team over the edge when planning too many and hence short iterations, during which the due dates follow each other too frequently. A great deal of empirical evidence from various domains shows robust links among overwork, exhaustion and turnover, and errors. It is reasonable, therefore, to suppose that, regardless of managerial prowess, team members remain susceptible to the dynamics underlying our study. Illusions of control are a poor substitute for decisions grounded in an understanding of how levels of agility shape schedule pressure and thereby affect learning, errors, and exhaustion, which is the larger implication of this study.

Appendix. Model Documentation

Overview of Model Structure

To be able to use the original model developed by Abdel-Hamid and Madnick (1991) to simulate agile software development, changes had to be made. In this model documentation, a list and brief description of all equations of the new model are given in Table A. To do this in a structured way, we have followed the reporting

guidelines for simulation-based research (Rahmandad & Sterman, 2012). In addition, Table B presents a list of all variables used in the model, together with the value of the exogenous variables (constants) and the source of the variables. The revised model (developed in iThink) is available from the authors upon request.

Table A. Formulation and Comments

Formulations and comments: Generic simulation and scenario parameters	Units
Initial time = 0	days
The initial time for the simulation.	
Final time = 300	days
The final time for the simulation.	
Time step: $dt = 1$	days
The time step for the simulation.	
Formulations and comments: Human resources	Units
$tw(t) = NHW(t) + EW(t)$	people
$NHW(t) = NHW(0) + \int_0^t (hr(s) - ar(s)) ds; NHW(0) = 0$	people
$EW(t) = EW(0) + \int_0^t (ar(s) - qr(s)) ds; EW(0) = 5$	people
$pex(t) = EW(t) / tw(t)$	dimensionless
This part of the model captures the hiring (hr), assimilation (ar), and quit rate (qr) of people. We segregate the project's total workforce (tw) into two employee types: newly hired workforce (NHW) and experienced workforce (EW). We make this distinction because new team members are usually less productive than veteran employees. This segregation also lets us capture the training process to assimilate new members. The veterans usually train the newcomers, both technically and socially. This is important, because this training can significantly affect a project's progress by reducing the veteran's productivity.	
$hr(t) = wr(t) / hd$	people/day
$wr(t) = EW(0) - (NHW(t) + EW(t))$	people
$ar(t) = NHW(t) / ad$	people/day

(continued)

Table A. (continued)

Formulations and comments: Human resources	Units
$qr(t) = \left(\frac{EW(t)}{et} \right) * me(t)$ $me(t) = \text{GRAPH}(E(t)) : (0, 1), (10, 1), (15, 1.05), (30, 1.3), (50, 3), (65, 6), (75, 9), (90, 15)$ Whenever a person leaves the team, this person is replaced (<i>wr</i>) to get the team size back to its initial level again. The hiring delay (<i>hd</i>) and assimilation delay (<i>ad</i>) slow down this process of replacing people. Team members can quit the project, either because of “normal attrition” after an average employment time (<i>et</i>) or because they are exhausted (<i>E</i>) by working overtime for too long (caused by schedule pressure). The relationship between exhaustion and turnover is non-linear and defined by the variable multiplier due to exhaustion (<i>me</i>).	people/day dimensionless
Formulations and comments: Experience and learning	Units
$EE(t) = EE(0) + \int_0^t (ieh(s) + iel(s) - edr(s) - dea(s)) ds; EE(0) = EW(0) * ree$ $ieh(t) = aexpn * hr(t)$ $iel(t) = dpaat(t)$ $dea(t) = qr(t) * aexpt(t)$ $aexpt(t) = EE(t) / tw(t)$ $ELQR(t) = ELQR(0) + \int_0^t dea(s) ds; ELQR(0) = 0$ $edr(t) = 0.5 * ipp(t) * pfr(t)$ $ELMA(t) = ELMA(0) + \int_0^t edr(s) ds; ELMA(0) = 0$ In the model of Abdel-Hamid and Madnick, experience is not explicitly modeled. In their model, newly hired and experienced employees have a nominal potential productivity (of respectively 0.5 and 1 tasks/person/day). This potential productivity does not gradually increase when new employees are trained and gain more experience, as we would expect in real-life. Therefore, we included an “Experience of Employees” (<i>EE</i>) stock in our model. Initially, experience is equal to the reference experience of employees (<i>ree</i>) times <i>EW</i> . At the start of a new project (or new iteration) this level of experience is relatively low and it gradually increases over time. Each new team member brings some experience to the team (increase of experience by hiring, <i>ieh</i>), but because the experience of new hires (<i>aexpn</i>) is lower than the currently average experience of the team (<i>aexpt</i>), bringing in a new team member decreases the average team experience. Otherwise, experience increases each day the team works on a development task and learns about the project (increase of experience by learning, <i>iel</i> , this will be explained later). Experience decreases through attrition (<i>dea</i>). Finally, experience decreases after an iteration (<i>edr</i>). When an iteration is finished, a project part is closed and the team starts with a new iteration, called a product in our model. At the start of this new iteration, the experience with respect to the new product that needs to be developed is low. Therefore, each time the team starts with a new iteration, the level of experience drops, after which it gradually increases with each day the team develops tasks. It is assumed that this drop in experience is equal to 50% of the experience that was gained during the process of developing the product.	person*days people people people days person*days person*days people person*days
$pel(t) = npe * \left(\frac{aexpt(t)}{ree} \right)^{elc}$ $sdp(t) = mclm(t) * pel(t)$ $mclm(t) = AFPP(t) * (1 - co(t))$ $co(t) = \text{GRAPH}(tw(t)) : (0, 0), (5, 0.015), (10, 0.06), (15, 0.135), (20, 0.24), (25, 0.375)$ The average experience of the team determines the productivity of employees (<i>pel</i>) via the learning curve (see also pp. 338 of Sterman, 2000). Software development productivity (<i>sdp</i>) is positively influenced by learning and negatively influenced by faulty processes (<i>mclm</i>). Productivity due to learning is the maximum level of productivity that can occur when an individual or group makes the best possible use of its resources. Losses from faulty processes are losses in productivity from factors such as communication and coordination overhead and low motivation. The relationship between team size and these faulty processes is non-linear (<i>co</i>).	tasks/(people*days) tasks/(people*days) tasks/(people*days) dimensionless
Formulations and comments: Allocation of human resources	Units
Staff is allocated to four different types of tasks that need to be done in a software development project. First, newly hired engineers need to be trained by the experienced ones. Second, software codes that are developed need to be reviewed (QA, quality assurance). Third, codes with errors need to be reworked. Finally, all remaining capacity will be devoted to software development (writing codes).	
$dpaat(t) = \text{IF}(pfr > 0) \text{ THEN } (0) \text{ ELSE } (tdp(t) - dpt(t))$ $dpt(t) = NHW(t) * tn$ The daily personpower available after training overhead (<i>dpaat</i>) is equal to the total daily personpower (<i>tdp</i>) minus the personpower that is spent on training (<i>dpt</i>), unless during the day an actual product is being delivered to the customer (product finish rate, <i>pfr</i>).	people people
$CPE(t) = CPE(0) + \int_0^t tdp(s) ds; CPE(0) = 0$	person*days

(continued)

Table A. (continued)

Formulations and comments: Allocation of human resources	Units
$tdp(t) = tw(t) * os(t)$	people
$CQAP(t) = CQAP(0) + \int_0^t dQAP(s)ds; CQAP(0) = 0$	person*days
$dQAP(t) = MIN\left(afbQA(t) * tdp(t), 0.9 * dpaat(t)\right)$	people
$afbQA(t) = dfbQA * \left(1 + papQA(t)\right)$	dimensionless
$papQA(t) = GRAPH\left(sp(t)\right) : (0, 0), (0.8, 0), (1, -0.005), (1.2, -0.025), (1.6, -0.35), (2, -0.5)$	dimensionless
The total daily personpower (tdp) that can be allocated to the project is equal to the total workforce times the overwork (os). Only a part of this tdp will be allocated on QA ($dQAP$), depending on the schedule pressure (sp) of the team.	
$CRP(t) = CRP(0) + \int_0^t drp(s)ds; CRP(0) = 0$	person*days
$drp(t) = MIN\left(decr(t) * rpe(t), dpp(t)\right)$	people
$dpp(t) = dpaat(t) - dQAP(t)$	people
$dsdp(t) = dpp(t) - drp(t)$	people
The daily rework personpower is defined by the minimum of the available daily production personpower (dpp) and the desired error correction rate ($decr$) times the personpower needed per error (npe). The daily production personpower is equal to the difference between $dpaat$ and $dQAP$, whereas the personpower available for software development ($dsdp$) is defined by the difference between dpp and drp .	
Formulations and comments: Software production	Units
Software production models the discovery of new tasks, the execution of tasks, the generation of errors while executing these tasks, and the discovery and rework of these errors. As software is developed, it is reviewed to detect any defects, for example, using quality assurance activities such as structured walkthroughs. Errors detected through such activities are reworked. Errors that are undetected can regenerate even more errors.	
$UT(t) = UT(0) + \int_0^t \left(utnpi(s) - drnt(s)\right)ds; UT(0) = rpdst(0) - ppdst(0)$	tasks
$utnpi(t) = psr(t) * \left(rpdst(t) - ppdst(t)\right)$	tasks/day
$drnt(t) = MAX\left(0, pdpd(t) * \left(UT(t) / 100\right)\right)$	tasks/day
$ppdpd(t) = GRAPH\left(ppdpc(t)\right) : (0, 0), (0.3, 1.2), (0.5, 3), (0.6, 4), (0.7, 6), (0.8, 8), (0.9, 54), (1, 100)$	tasks/day
The level of undiscovered tasks (UT) initially equals the difference between the real and perceived product size in tasks ($rpdst, ppdst$). The level is increased when a new iteration starts ($utnpi$) and decreased when these tasks are discovered ($drnt$). This discovery rate is influenced by the percentage of discoveries per day ($ppdpd$), which in turn depends on the percentage of the product that is perceived complete ($ppdpc$).	
$TD(t) = TD(0) + \int_0^t \left(drnt(s) + tnpi(s) - tdr(s)\right)ds; TD(0) = ppdst(0)$	tasks
$tnpi(t) = psr(t) * ppdst(t)$	tasks/day
$tdr(t) = MIN\left(TD(t) / m_{dt}, dsdp(t) * sdpr(t)\right)$	tasks/day
Tasks that have to be developed (TD) are accumulated through the $drnt$, the tasks of the new product in iteration ($tnpi$) minus the tasks development rate (tdr). The tdr can never be larger than available capacity ($dsdp * sdpr$).	
$TF(t) = TF(0) + \int_0^t \left(tdr(s) - opdf(s)\right)ds; TF(0) = 0$	tasks
$opdf(t) = IF(pfr > 0) THEN \left(TF(t) / dt\right) ELSE 0$	tasks/day
$CTF(t) = CTF(0) + \int_0^t opdf(s)ds; CTF(0) = 0$	tasks
The level of tasks finished (TF) calculates how many tasks of the current iteration are finished. The stock depletes entirely when the product is finished ($opdf$), in that case all the finished tasks flow to the stock of cumulative tasks finished (CTF).	
$apjpc(t) = ctd(t) / rpjst$	dimensionless
$ctd(t) = TF(t) + CTF(t)$	tasks
$cppst(t) = TD(t) + TF(t)$	tasks
$apdpc(t) = \frac{TF(t)}{rpdst * MAX\left(0.001, Pil(t)\right)}$	dimensionless
The percentage of the project that is actually completed ($apjpc$) is defined by the cumulative tasks developed (ctd) and the real project size in tasks ($rpjst$). The currently perceived product size in tasks ($cppst$) equals the stocks of TD and TF . The percentage of the product that is actually completed ($apdpc$) is influenced by the ratio of TF and the real product size in tasks ($rpdst$).	

(continued)

Table A. (continued)

Formulations and comments: Software production	Units
$UE(t) = UE(0) + \int_0^t (egr(s) - erdr(s)) ds; UE(0) = 0$ $egr(t) = tdr(t) * ept(t) + err(t)$ $ept(t) = net(t) * megrsp(t) * megpe(t)$ $net(t) = neKDSI(t) * DSI / 1000$ $neKDSI(t) = GRAPH(apdpc(t)) : (0, 27), (0.2, 25.9), (0.4, 23.6), (0.6, 17.9), (0.8, 15.6), (1, 14.5)$ $megrsp(t) = GRAPH(sp(t)) : (-0.4, 0.9), (0, 1), (0.6, 1), (1, 1.14), (1.6, 1.5), (2, 1.9), (2.2, 2)$ $megpe(t) = GRAPH(ipe(t)) : (0, 2), (0.2, 1.8), (0.4, 1.6), (0.6, 1.4), (0.8, 1.2), (1, 1)$ $ipe(t) = (MIN(1, aexpt(t)/aexpt(0)) + pex(t)) / 2$ Errors are generated (<i>egr</i>) when new tasks are developed. The number of errors per task (<i>ept</i>) depends on the percentage of the product that is complete (<i>apdpc</i>), the schedule pressure (<i>megrsp</i>) and the experience of the team (<i>megpe</i>). The higher the percentage complete, the lower the schedule pressure and the higher the experience, the lower the number of errors per task will be.	errors errors/day errors/task errors/task errors/DSI dimensionless dimensionless dimensionless
$err(t) = UE(t) * erp(t) / 100$ $erp(t) = GRAPH(ppdpc(t)) : (0, 0.5), (0.2, 0.45), (0.4, 0.375), (0.6, 0.2), (0.8, 0.075), (1, 0)$ $ppdpc(t) = TF(t) / MAX(1, cppst(t))$ $erdr(t) = MIN(UE(t), dQAp(t)/QApe(t)) / mdt$ $QApe(t) = (nQApe(t) / mclm(t)) * mdeed(t)$ $nQApe(t) = GRAPH(cde(t)/mep(t)) : (0, 0.3), (0.2, 0.32), (0.4, 0.34), (0.6, 0.4), (0.8, 0.5), (1, 0.7)$ $cde(t) = DE(t) + CER(t)$ $mep(t) = (rpjsDSI / 1000) * meKDSI$ $mdeed(t) = GRAPH(edKDSI(t)) : (0, 50), (0.2, 26), (0.4, 10), (0.6, 1.75), (0.8, 1), (1, 1)$ $edKDSI(t) = ept(t) * 1000 / DSI$ Errors can regenerate (<i>err</i>) other errors as long as they are undetected. This depends on the error regeneration percentage (<i>erp</i>) which in turn depends on the perceived percentage of the product that is complete (<i>ppdpc</i>). The error detection rate (<i>erdr</i>) depends on the capacity allocated to QA (<i>dQAp</i>) and the capacity required to detect an error (<i>QApe</i>). More capacity is required when more errors have been detected already. In addition, the higher the error density, the more capacity is required to detect errors (<i>mdeed</i>).	errors/day dimensionless/day dimensionless errors/day people/error people/error errors errors dimensionless errors/DSI
$DE(t) = DE(0) + \int_0^t (erdr(s) - rr(s)) ds; DE(0) = 0$ $rr(t) = MIN(DE(t), drp(t)/rpe(t)) / mdt$ $rpe(t) = nrpe(t) / mclm(t)$ $nrpe(t) = GRAPH(ppdpc(t)) : (0, 0.725), (0.2, 0.69), (0.4, 0.6), (0.6, 0.48), (0.8, 0.39), (1, 0.36)$ The rework rate of errors (<i>rr</i>) is influenced by the minimum of the number of detected errors (<i>DE</i>) and the ratio of capacity allocated to rework (<i>drp</i>) and capacity required for rework (<i>rpe</i>). Capacity required for rework is in turn influenced by the perceived completeness of the product (<i>ppdpc</i>) and communication overhead (<i>mclm</i>).	errors errors/day people/error people/error
$CER(t) = CER(0) + \int_0^t rr(s) ds; CER(0) = 0$ $tepj(t) = UE(t) + DE(t) + CER(t)$ $decr(t) = DE(t) / drd(t)$ $drd(t) = GRAPH(ppdpc(t)) : (0, 15), (0.2, 15), (0.4, 15), (0.6, 14), (0.8, 8), (1, 1)$ Finally, the desired error correction rate (<i>decr</i>), which is used for the resource allocation decision (<i>drp</i>), depends on the number of detected errors (<i>DE</i>) and the desired rework delay (<i>drd</i>) which in turn depends on the completeness of the product (<i>ppdpc</i>): the higher the completeness, the shorter the desired delay.	errors errors errors dimensionless
Formulations and comments: Planning	Units
$PiB(t) = PiB(0) + \int_0^t -psr(s) ds; PiB(0) = LoA - 1$ $psr(t) = IF(PiI(t) < 0.001) THEN (MIN(1, PiB(t)/dt), ELSE(0))$	dimensionless dimensionless/day

(continued)

Table A. (continued)

Formulations and comments: Planning	Units
$pfr(t) = IF(LoA = 1) THEN (IF(apdpc(t) * 100 \geq 99.5), THEN(Pil(t)/dt), ELSE(0)),$ $ELSE(IF(ROUND(apdpc(t) * 100) \geq 99.5), THEN(Pil(t)/dt), ELSE(0))$	dimensionless/day
$dfr1(t) = DELAY(pfr(t), dt, 0)$	dimensionless/day
$dfr2(t) = DELAY(pfr(t), 2 * dt, 0)$	dimensionless/day
$Pil(t) = Pil(0) + \int_0^t (psr(s) - pfr(s)) ds; Pil(0) = 1$	dimensionless
$PR(t) = PR(0) + \int_0^t pfr(s) ds; PR(0) = 0$	dimensionless
Agile software development implies that the entire project is executed in several iterations. The higher the level of agility (<i>LoA</i>), the more iterations the project has. Each iteration delivers a part of the project. We call these parts products. There is always one product that is developed in an iteration (<i>Pil</i>). Thus, the level of agility indicates the number of products in the project. For a sequential project, the <i>LoA</i> is equal to 1. When a product (iteration) is finished (<i>pfr</i>), a new product can be taken out of the backlog (<i>PiB</i>) and a new iteration starts (<i>psr</i>). When a product is finished it flows into the stock of products reviewed (<i>PR</i>).	
$rpds = rpdsDSI / DSIt$	tasks
$rpdsDSI = rpjsDSI / LoA$	DSI
$rpjst = rpjsDSI / DSIt$	tasks
$ppjsDSI = (1 - uf) * rpjsDSI$	DSI
$ppjst = ppjsDSI / DSIt$	tasks
$ppdst = ppjst / LoA$	tasks
$ipp = itp / LoA$	person*days
$trpi(t) = pidd(t) - t$	days
$pidd(t) = (pjdd / LoA) * Pil(t) + (pjdd / LoA) * PR(t)$	days
$pdd(t) = MAX(pjdd, (tppn(t) / tw(t)) + t)$	days
When the level of agility is 5, for example, <i>LoA</i> is 5. All five products are assumed to be of equal size, determined by the total project size (<i>rpjsDSI</i>) divided by 5. At $t = 1$, the model starts simulating the development of the first part or product. Because <i>rpjsDSI</i> is equal to 24400, the real size of one product (<i>rpdsDSI</i>) in an Agile-5 project is $24400/5 = 4880$ DSI. Likewise, the initial number of persondays required to finish one product (<i>ipp</i>) is defined by the ratio of the initial total number of persondays in the project (<i>itp</i>) and the <i>LoA</i>. The same logic applies for determining the due date of the product in iteration (<i>pidd</i>), which is influenced by the due date of the project (<i>pjdd</i>) and the <i>LoA</i>. The perceived project size (<i>ppjsDSI</i>) depends on the uncertainty fraction (<i>uf</i>). Finally, the projected due date (the forecasted due date based on the progress of the project, <i>pdd</i>) is determined by the total persondays perceived still needed (<i>tppn</i>), the total workforce (<i>tw</i>) and the current time (<i>t</i>).	
Formulations and comments: Control	Units
$CSP(t) = CSP(0) + \int_0^t sp(s) ds; CSP(0) = 0$	dimensionless*day
$sp(t) = IF(dfr1(t) > 0) THEN(1), ELSE(MIN(2.5, tppn(t) / MAX(1, prt(t))))$	dimensionless
$asp(t) = CSP(t) / MAX(1, t)$	dimensionless
$tppn(t) = ppnnt(t) + ppnr(t)$	person*days
$ppnnt(t) = TD(t) / pe(t)$	person*days
$ppnr(t) = DE(t) * rpe(t) * mdt$	person*days
Schedule pressure (<i>sp</i>) equals the ratio of the total persondays perceived still needed (<i>tppn</i>) and the persondays remaining in time. It is assumed that when pressure is higher than 2.5, the effect on the team is the same as when it is 2.5. Average schedule pressure (<i>asp</i>) is the ratio of the cumulative schedule pressure (<i>CSP</i>) and the current time (<i>t</i>). The <i>tppn</i> is determined by the persondays needed for tasks (<i>ppnnt</i>) and rework (<i>ppnr</i>). The former is influenced by the tasks to be developed (<i>TD</i>) and the productivity estimate (<i>pe</i>). The latter is influenced by the detected errors (<i>DE</i>) and the rework personpower needed per error (<i>rpe</i>).	
$pe(t) = pf(t) * sdp(t) + (1 - pf(t)) * pdp(t)$	tasks/(person*days)
$pf(t) = GRAPH(ppdpc(t)) : (0, 1), (0.2, 0.8), (0.4, 0.6), (0.6, 0.4), (0.8, 0.2), (1, 0)$	dimensionless
$pdp(t) = MAX(0.25, ctd(t) / MAX(1, CPE(t)))$	tasks/(person*days)
In the beginning of the iteration, the productivity estimate (<i>pe</i>) is primarily determined by the software development productivity (<i>sdp</i>), at the end of the iteration, the perceived development productivity (<i>pdp</i>) is more important. This varying influence is defined by the productivity factor (<i>pf</i>).	
$psp(t) = tppn(t) - MIN(tppn(t), prt(t))$	person*days
$prt(t) = MAX(0, (pidd(t) - t) * tw(t))$	person*days

(continued)

Table A. (continued)

Formulations and comments: Control	Units
$TPSP(t) = TPSP(0) + \int_0^t raps(s)ds; TPSP(0) = ipp$	person*days
$raps(t) = (tpbn(t) + CPE(t) - TPSP(t)) / daps$	person
The perceived shortage in persondays (<i>psp</i>) is the difference between the total persondays perceived still needed (<i>tpbn</i>) and the persondays remaining in time (<i>prt</i>). This <i>prt</i> is defined by the due date of the current iteration (<i>pidd</i>), the current time (<i>t</i>), and the total workforce (<i>tw</i>). The total project size in persondays (<i>TPSP</i>) accumulates the rate of adjusting project size (<i>raps</i>), which is influenced by the total persondays perceived still needed (<i>tpbn</i>) and the cumulative persondays expended (<i>CPE</i>).	
$AFPP(t) = CSP(0) + \int_0^t far(s)ds; AFPP(0) = nfpp$	dimensionless
$far(t) = (wrs(t) - AFPP(t)) / fad(t)$	dimensionless/day
$fad(t) = nad(t) * ewrs(t)$	days
$nad(t) = GRAPH(trpi(t)) : (0, 1.5), (5, 1.5), (10, 2), (20, 2.7), (30, 3.5), (40, 5)$	days
$ewrs(t) = IF(wrs(t) \geq AFPP(t)), THEN(1), ELSE(0.75)$	dimensionless
$wrs(t) = (1 + pbws(t)) * nfpp$	dimensionless
$os(t) = GRAPH(wrs(t)) : (0.6, 1), (0.64, 1.04), (0.68, 1.08), (0.72, 1.12), (0.76, 1.16), (0.8, 1.2)$	dimensionless
The actual fraction of a personday that a person spends on a project (<i>AFPP</i>) is initially equal to the nominal fraction (<i>nfpp</i>), but can be adjusted (<i>far</i>) over time (<i>fad</i>). This adjustment is given by the workrate sought (<i>wrs</i>). The <i>fad</i> is equal to the ratio of the normal adjustment delay (<i>nad</i>) and the effect of workrate sought (<i>ewrs</i>). The former is determined by the time remaining for the product in iteration (<i>trpi</i>). The <i>ewrs</i> makes the <i>fad</i> longer when the team needs to increase their <i>AFPP</i> and shorter otherwise. The <i>wrs</i> is influenced by the percentage of boost in workrate sought (<i>pbws</i>). In turn, the <i>wrs</i> influences the overwork by staff (<i>os</i>).	
$pbws(t) = IF(psp(t) \geq 0), THEN(hp(t) / (tw(t) * odt(t))),$ $ELSE(hp(t) / (tpbn(t) - hp(t) + 1))$	dimensionless
$hp(t) = IF(psp(t) \geq 0), THEN(MIN(mpsh(t), psp(t))), ELSE(-psp(t))$	person*days
$mpsh(t) = odt(t) * tw(t) * wo(t)$	person*days
$odt(t) = nodt(t) * modt(t)$	days
$nodt(t) = GRAPH(trpi(t)) : (0, 1), (10, 5), (20, 15), (30, 25), (40, 35), (50, 35)$	days
$modt(t) = GRAPH(E(t)/mte) : (0, 1), (0.2, 0.8), (0.4, 0.6), (0.6, 0.4), (0.8, 0.2), (1, 0)$	dimensionless
$wo(t) = IF(t \geq TLEB(t) + TDe(t)), THEN(1), ELSE(0)$	dimensionless
The percentage of boost in work rate sought (<i>pbws</i>) is primarily depending on the persondays the team can handle (<i>hp</i>) with respect to working overtime. If the perceived shortage in persondays (<i>psp</i>) is larger than 0, the team realizes it needs to adjust their work rate by, for example, working overtime. However, the team can only work overtime for a certain amount of time due to fatigue and exhaustion. The maximum number of persondays shortage handled (<i>mpsh</i>) is influenced by the overwork duration threshold (<i>odt</i>), the workforce (<i>tw</i>) and the willingness to overwork (<i>wo</i>). The team is willing to overwork when it has recovered from the last exhaustion breakdown, which is influenced by the time of the last exhaustion breakdown (<i>TLEB</i>) and the time to de-exhaust (<i>TDe</i>).	
$E(t) = E(0) + \int_0^t (rie(s) - rde(s))ds; E(0) = 0$	dimensionless
$rie(t) = GRAPH\left(\frac{1 - AFPP(t)}{1 - nfpp}\right) : (-0.5, 2.5), (-0.2, 1.6), (0, 1.15), (0.1, 0.9), (0.5, 0.5), (0.9, 0)$	dimensionless/day
$rde(t) = IF(0 \geq rie(t)), THEN(E(t)/edd), ELSE(0)$	dimensionless/day
Exhaustion (<i>E</i>) increases every day the team works harder than normal (<i>rie</i>) and decreases when the team does not work harder than normal (<i>rde</i>).	
$TDe(t) = TDe(0) + \int_0^t rctd(s)ds; TDe(0) = 0$	days
$rctd(t) = IF(E(t)/mte \geq 0.1), THEN(1), ELSE(-TDe(t)/mtd)$	days/day
$TLEB(t) = TLEB(0) + \int_0^t ctdeb(s)ds; TLEB(0) = 0$	days
$ctdeb(t) = IF(odt(t) = 0), THEN((t - TLEB(t))/dt), ELSE(0)$	days/day
The time to de-exhaust (<i>TDe</i>) increases when exhaustion starts to build up, and it decreases when exhaustion is really low, indicated by the rate to control time to de-exhaust (<i>rctd</i>). The time of the last exhaustion breakdown (<i>TLEB</i>) is the accumulation of the change time of the last exhaustion breakdown (<i>ctdeb</i>). This <i>ctdeb</i> is influenced by the overwork duration threshold (<i>odt</i>) and the <i>TLEB</i> .	

Table B. List of All Variables

Abbreviation	Variable Name	Value	Units	Source
ad	assimilation delay	80	days	1
aexpn	average experience of new hires	0.2	days	2
aexpt	average experience of team member		days	2
AFPP	Actual Fraction of Personday on Project		dimensionless	1
afpQA	actual fraction of personpower for QA		dimensionless	1
apdpc	actual product% complete		dimensionless	2
apjpc	actual project% complete		dimensionless	2
ar	assimilation rate		persons/day	1
asp	average schedule pressure		dimensionless	2
cde	cumulative number of detected errors		errors	1
CER	Cumulative Errors Reworked		errors	1
CER(0)	Cumulative Errors Reworked(0) ((0) refers to initial value)	0	errors	1
co	communication overhead		dimensionless	1
CPE	Cumulative Persondays Expended		person*days	1
CPE(0)	Cumulative Persondays Expended(0)	0	person*days	1
cppst	currently perceived product size in tasks		tasks	2
CQAP	Cumulative QA Persondays(t)		person*days	1
CQAP(0)	Cumulative QA Persondays(0)	0	person*days	1
CRP	Cumulative Rework Persondays		person*days	1
CRP(0)	Cumulative Rework Persondays(0)	0	person*days	1
CSP	Cumulative Schedule Pressure		dimensionless*days	2
CSP(0)	Cumulative Schedule Pressure(0)	0	dimensionless*days	2
ctd	cumulative tasks developed		tasks	1
CTF	Cumulative Tasks Finished		tasks	1
CTF(0)	Cumulative Tasks Finished(0)	0	tasks	1
ctleb	change time of last exhaustion breakdown		days/day	1
daps	delay in adjusting project size	1	day	1
DE	Detected Errors		errors	1
DE(0)	Detected Errors(0)	0	errors	1
dea	decrease of experience by attrition		people	2
decr	desired error correction rate		errors/day	1
dfpQA	desired fraction of personpower on QA	0.2	dimensionless	1
dfr1	delay finish rate 1		dimensionless/day	2
dfr2	delay finish rate 2		dimensionless/day	2
dpaat	daily personpower available after training overhead		people	1
dpp	daily production personpower		people	1
dpt	daily personpower for training		people	1
dQAp	daily QA personpower		people	1
drd	desired rework delay		dimensionless	1
drnt	discovery rate new tasks		tasks/day	1
drp	daily rework personpower		people	1
dsdp	daily SD personpower		people	1
DSIt	DSI per task	60	DSI/task	1
dt	time step of the simulation	1	day	1
E	Exhaustion		dimensionless	1
E(0)	Exhaustion(0)	0	dimensionless	1
edd	exhaustion depletion delay time	20	days	1
edKDSI	error density per KDSI		errors/DSI	1
edr	experience decay rate		people	2
EE	Experience of Employees		person*days	2
EE(0)	Experience of Employees(0)		person*days	2
egr	error generation rate		errors/day	1
elc	exponent learning curve	0.3219	dimensionless	1
ELMA	Experience Lost by Mental Accounting		person*days	2
ELMA(0)	Experience Lost by Mental Accounting(0)	0	person*days	2
ELQR	Experience Lost by Quit Rate		person*days	2
ELQR(0)	Experience Lost by Quit Rate(0)	0	person*days	2
ept	errors per task		errors/task	1

(continued)

Table B. (continued)

Abbreviation	Variable Name	Value	Units	Source
erdr	error detection rate		errors/day	1
erp	error regeneration%		dimensionless/day	1
err	error regeneration rate		errors/day	1
et	employment time	700	days	1
EW	Experienced Workforce		people	1
EW(0)	Experienced Workforce(0)	4.9	people	1
ewrs	effect of workrate sought		dimensionless	1
fad	fraction adjustment delay		days	1
far	fraction adjustment rate		dimensionless/day	1
hd	hiring delay	40	days	1
hp	handled persondays		person*days	1
hr	hiring rate		people/day	1
ieh	increase of experience by hiring		people	2
iel	increase of experience by learning		people	2
ipe	indicator of %experience		dimensionless	2
ipp	initial persondays in product		person*days	2
itp	initial total persondays	945	person*days	1
LoA	Level of Agility (number of products/iterations in project)	1	dimensionless	2*
mclm	multiplier due to communication losses and motivation		dimensionless	1
mdeed	multiplier to detection effort due to error density		dimensionless	1
mdt	minimum development time	1	day	2
me	multiplier due to exhaustion		dimensionless	1
megpe	multiplier to error generation due to %experience		dimensionless	1
megsp	multiplier to error generation due to schedule pressure		dimensionless	1
meKDSI	maximum number of errors made per KDSI	25	errors/DSI	1
mep	maximum number of errors per project		errors	1
modt	multiplier to overwork duration threshold		dimensionless	1
mpsh	max persondays shortage handled		person*days	1
mte	maximum tolerable exhaustion	45	dimensionless	1
nad	normal adjustment delay		days	1
neKDSI	nominal errors per KDSI		errors/DSI	1
net	nominal errors per task		errors/task	1
nfpp	nominal fraction of personday on project	0.6	dimensionless	1
NHW	Newly Hired Workforce		people	1
NHW(0)	Newly Hired Workforce(0)	0	people	1
nodt	nominal overwork duration threshold		days	1
npe	nominal productivity of employees	0.8	tasks/(person*days)	1
nQApe	nominal QA personpower needed per error		people/errors	1
nrpe	nominal rework personpower needed per error		people/errors	1
odt	overwork duration threshold		days	1
opdf	outflow when product is finished		tasks/day	2
os	overwork by staff		dimensionless	1
papQA	percent adjustment in planned fraction of personpower for QA		dimensionless	1
pbws	percentage boost in workrate sought		dimensionless	1
pdd	projected due date		days	1
pdp	perceived development productivity		tasks/(person*days)	1
pdpd	percentage of discoveries per day		dimensionless/day	1
pe	productivity estimate		tasks/(person*days)	1
pel	productivity of employees due to learning		tasks/(person*days)	1
pex	percentage experienced		dimensionless	2
pf	productivity factor		dimensionless	1
pfr	product finish rate		dimensionless/day	2
PiB	Products in Backlog		dimensionless	2
PiB(0)	Products in Backlog(0)		dimensionless	2
pidd	product in iteration due date		days	2
pii	Products in Iteration		dimensionless	2
Pil	Products in Iteration		dimensionless	2
Pil(0)	Products in Iteration(0)	1	dimensionless	2

(continued)

Table B. (continued)

Abbreviation	Variable Name	Value	Units	Source
pjdd	project due date	260	days	1
ppdpc	perceived product% complete		dimensionless	2
ppdst	perceived product size in tasks		tasks	2
ppjsDSI	perceived project size in DSI		DSI	1
ppjst	perceived project size in tasks		tasks	1
ppnnt	persondays perceived still needed for new tasks		person*days	1
ppnr	persondays perceived still needed for rework of detected errors		person*days	1
PR	Products Reviewed		dimensionless	2
PR(0)	Products Reviewed(0)	0	dimensionless	2
prt	persondays remaining in time		person*days	1
psp	perceived shortage in persondays		person*days	1
psr	product start rate		dimensionless/day	2
QApe	QA personpower needed per error		people/error	1
qr	quit rate		people/day	1
raps	rate of adjusting project size		people	1
rctd	rate to control time to deexhaust		days/day	1
rde	rate of decrease of exhaustion		dimensionless/day	1
ree	reference experience of employees	548	days	2
rie	rate of increase of exhaustion		dimensionless/day	1
rpdsDSI	real product size in DSI		DSI	2
rpdst	real product size in tasks		tasks	2
rpe	rework personpower needed per error		people/error	1
rpjsDSI	real project size in DSI	24400	DSI	1
rpjst	real project size in tasks		tasks	1
rr	rework rate		errors/day	1
sdp	software development productivity		tasks/(people*day)	1
sp	schedule pressure		dimensionless	1
TD	Tasks to be Developed		tasks	1
TD(0)	Tasks to be Developed(0)		tasks	1
TDe	Time to Deexhaust(t)		days	1
TDe(0)	Time to Deexhaust(0)	0	days	1
tdp	total daily personpower		people	1
tdr	tasks development rate		tasks/day	1
tepj	total number of errors in project		errors	1
TF	Tasks Finished		tasks	1
TF(0)	Tasks Finished(0)	0	tasks	1
TLEB	Time of Last Exhaustion Breakdown		days	1
TLEB(0)	Time of Last Exhaustion Breakdown(0)	0	days	1
tn	trainers per newly hired staff	0.2	dimensionless	1
tnpi	tasks of new product in iteration		tasks/day	2
tppn	total persondays perceived still needed		person*days	1
TPSP	Total Project Size in Persondays		person*days	1
TPSP(0)	Total Project Size in Persondays(0)		person*days	1
trpi	time remaining for product in iteration		days	2
tw	total workforce		people	1
UE	Undetected Errors(t)		errors	1
UE(0)	Undetected Errors(0)	0	errors	1
uf	uncertainty fraction	0.35	dimensionless	1
UT	Undiscovered Tasks(t)		tasks	1
UT(0)	Undiscovered Tasks(0)		tasks	1
utnpi	undiscovered tasks of new product in iteration		tasks/day	2
wo	willingness to overwork		dimensionless	1
wr	workforce to replace		people	1
wrs	workrate sought		dimensionless	1

Source descriptions:

1 This variable is taken from the model developed by Abdel-Hamid and Madnick (1991).

2 This variable is included in the model by the authors to include agile processing.

2* This variable determines which scenario will be modeled (level of agility).

Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) received no financial support for the research, authorship, and/or publication of this article.

References

- Abdel-Hamid, T. K. (1993). A multiproject perspective of single-project dynamics. *Journal of Systems Software*, 22(3), 151–166.
- Abdel-Hamid, T. K. (2010). Single-loop project controls: Reigning paradigms or straitjackets? *Project Management Journal*, 42(1), 17–30.
- Abdel-Hamid, T. K., & Madnick, S. E. (1991). *Software project dynamics—An integrated approach*. Englewood Cliffs, NJ: Prentice-Hall.
- Abdel-Hamid, T. K., Sengupta, K., & Swett, C. (1999). The impact of goals on software project management: An experimental investigation. *MIS Quarterly*, 23(4), 531–555.
- Ågerfalk, P. J., Fitzgerald, B., & Slaughter, S. A. (2009). Flexible and distributed information systems development: State of the art and research challenges. *Information Systems Research*, 20(3), 317–328.
- Agrawal, M., & Chari, K. (2007). Software effort, quality, and cycle time: A study of CMM level 5 projects. *IEEE Transactions on Software Engineering*, 33(3), 145–156.
- Akkermans, H., & van Oorschot, K. E. (2016). Pilot error? Managerial decision biases as explanation for disruptions in aircraft development. *Project Management Journal*, 47(2), 79–102.
- Amabile, T. M. (1988). A model of creativity and innovation in organizations. In B. M. Staw & L. L. Cummings (Eds.), *Research in organizational behavior* (Vol. 10, pp. 123–167). Greenwich, CT: JAI Press.
- Amabile, T. M., Hadley, C. N., & Kramer, S. J. (2002). Creativity under the gun. *Harvard Business Review*, 80(8), 52–63.
- Ancona, D., Okhuysen, A., & Perlow, L. (2001). Taking time to integrate temporal research. *Academy of Management Review*, 26(4), 512–529.
- Argote, L. (1999). *Organizational learning: Creating, retaining, and transferring knowledge*. Norwell, MA: Kluwer.
- Austin, R. D. (2001). The effects of time pressure on quality in software development: An agency model. *Information Systems Research*, 12(2), 195–207.
- Austin, R. D., & Devin, L. (2009). Weighing the benefits and costs of flexibility in making software: Toward a contingency theory of the determinants of development process design. *Information Systems Research*, 20(3), 462–477.
- Baer, M., & Oldham, G. R. (2006). The curvilinear relation between experienced creative time pressure and creativity: Moderating effects of openness to experience and support for creativity. *The Journal of Applied Psychology*, 91(4), 963–970.
- Bjarnason, E., Wnuk, K., & Regnell, B. (2012). Are you biting off more than you can chew? A case study on causes and effects of overscoping in large-scale software engineering. *Information and Software Technology*, 54(10), 1107–1124.
- Boehm, B. W. (1981). *Software engineering economics*. Englewood Cliffs, NJ: Prentice-Hall.
- Boehm, B. W., & Turner, R. (2005). Management challenges to implementing agile processes in traditional development organizations. *IEEE Software*, 22(5), 30–39.
- Boh, W. F., Slaughter, S. A., & Espinosa, J. A. (2007). Learning from experience in software development: A multilevel analysis. *Management Science*, 53(8), 1315–1331.
- Buehler, R. D., Griffin, D., & Ross, M. (1994). Exploring the “planning fallacy:” Why people underestimate their task completion times. *Journal of Personality and Social Psychology*, 67(3), 366–381.
- Buehler, R., Griffin, D., & Ross, M. (2002). Inside the planning fallacy: The causes and consequences of optimistic time predictions. In T. Gilovich, D. Griffin, & D. Kahneman (Eds.), *Heuristics and biases: The psychology of intuitive judgment*. Cambridge, UK: Cambridge University Press.
- Cao, L., Ramesh, B., & Abdel-Hamid, T. (2010). Modeling dynamics in agile software development. *ACM Transactions on Management Information Systems*, 1(1), 1–26.
- Conboy, K. (2009). Agility from first principles: Reconstructing the concept of agility in information systems development. *Information Systems Research*, 20(3), 329–354.
- Conforto, E. C., Salum, F., Amaral, D. C., Da Silva, S. L., & de Almeida, L. F. M. (2014). Can agile project management be adopted by industries other than software development? *Project Management Journal*, 45(3), 21–34.
- Cooper, R. G. (1994). Perspective third-generation new product processes. *Journal of Product Innovation Management*, 11(1), 3–14.
- Cooper, R. G. (2008). Perspective: The Stage-Gate® idea-to-launch process—Update, what’s new, and NexGen systems. *Journal of Product Innovation Management*, 25(3), 213–232.
- Cronin, M., Gonzalez, C., & Sterman, J. (2009). Why don’t well-educated adults understand accumulation? *Organizational Behavior and Human Decision Processes*, 108(1), 116–130.
- Davis, J. P., Eisenhardt, K. M., & Bingman, C. B. (2007). Developing theory through simulation methods. *Academy of Management Review*, 32(2), 480–499.
- Dawson, R. J., & Dawson, C. W. (1998). Practical proposals for managing uncertainty and risk in project planning. *International Journal of Project Management*, 16(5), 299–310.
- Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. (2012). A decade of agile methodologies: Towards explaining agile software development. *The Journal of Systems and Software*, 85(6), 1213–1221.
- Drury, M., Conboy, K., & Power, K. (2012). Obstacles to decision making in agile software development teams. *Journal of Systems and Software*, 85(6), 1239–1254.
- Dutta, A. (2001). Business planning for network services: A systems thinking approach. *Information Systems Research*, 12(3), 260–283.
- Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9–10), 833–859.
- Eisenhardt, K. M., & Tabrizi, B. N. (1995). Accelerating adaptive processes: Product innovation in the global computer industry. *Administrative Science Quarterly*, 40, 84–110.

- Ellis, A. P. J. (2006). System breakdown: The role of mental models and transactive memory in the relationship between acute stress and team performance. *Academy of Management Journal*, 49(3), 576–589.
- Ericksen, J., & Dyer, L. (2004). Right from the start: Exploring the effects of early team events on subsequent project team development and performance. *Administrative Science Quarterly*, 49(3), 438–471.
- Fitzgerald, B., Hartnett, G., & Conboy, K. (2006). Customising agile methods to software practices. *European Journal of Information Systems*, 15(2), 197–210.
- Ford, D. N., & Sterman, J. D. (1998). Dynamic modeling of product development processes. *System Dynamics Review*, 14(1), 31–68.
- Fowler, M., & Highsmith, J. (2001). The agile manifesto. *Software Development*, 9, 28–32.
- Fruhling, A., & de Vreede, G. J. (2006). Field experiences with eXtreme programming: Developing an emergency response system. *Journal of Management Information Systems*, 22(4), 39–68.
- Gardner, H. K. (2012). Performance pressure as a double-edged sword: Enhancing team motivation but undermining the use of team knowledge. *Administrative Science Quarterly*, 57(1), 1–46.
- Gevers, J. M. P., van Eerde, W., & Rutte, C. G. (2001). Time pressure, potency, and progress in project groups. *European Journal of Work and Organizational Psychology*, 10(2), 205–221.
- Glaier, F., Moulton, A., & Madnick, S. (2013, July 21–25). *Agile project dynamics: A system dynamics investigation of agile software development methods*. In Proceeding of 31st International Conference of the System Dynamics Society, Cambridge, MA.
- Glickman, S., & Kopcho, J. (1995). *Bellcore's experiences using Abdel-Hamid's systems dynamics model*. COCOMO Conference, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA.
- Goldratt, E. (1997). *Critical chain*. Great Barrington, MA: The North River Press.
- Harris, M. L., Collins, R. W., & Hevner, A. R. (2009). Control of flexible software development under uncertainty. *Information Systems Research*, 20(3), 400–419.
- Homer, J. (2014). Levels of evidence in system dynamics modeling. *System Dynamics Review*, 30(1–2), 75–80.
- Iansiti, M., & MacCormack, A. (1997). Developing products on internet time. *Harvard Business Review*, 75(5), 108–117.
- Karau, S. J., & Kelly, J. R. (1992). The effects of time scarcity and time abundance on group performance quality and interaction process. *Journal of Experimental Social Psychology*, 28(6), 542–571.
- Kelly, J., & Karau, S. (1999). Group decision making: The effects of initial preferences and time pressure. *Personality and Social Psychology Bulletin*, 25(11), 1342–1354.
- Kessler, E. H., & Bierly, P. E. (2002). Is faster really better? An empirical test of the implications of innovation speed. *IEEE Transactions on Engineering Management*, 49(1), 2–12.
- Larman, C. (2006). Agile and iterative development: A manager's guide. In A. Cockburn & J. Highsmith (Eds.), *Agile software development series*. Boston, MA: Addison-Wesley Publishing Company.
- Latham, G. P., & Seijts, G. H. (1999). The effects of proximal and distal goals on performance on a moderately complex task. *Journal of Organizational Behavior*, 20(4), 421–429.
- Lee, G., & Xia, W. (2010). Toward agile: An integrated analysis of quantitative and qualitative field data on software development agility. *MIS Quarterly*, 34(1), 87–114.
- Lindstrom, L., & Jeffries, R. (2004). Extreme programming and agile software development methodologies. *Information Systems Management*, 21(3), 41–60.
- Lyneis, J. M., & Ford, D. N. (2007). System dynamics applied to project management: A survey, assessment, and directions for future research. *System Dynamics Review*, 23(2–3), 157–189.
- Marks, M., Mathieu, J., & Zaccaro, S. (2001). A temporally based framework and taxonomy of team processes. *Academy of Management Review*, 26(3), 356–376.
- Maruping, L. M., Venkatesh, V., & Agarwal, R. (2009). A control theory perspective on agile methodology use and changing user requirements. *Information Systems Research*, 20(3), 377–399.
- Maruping, L. M., Venkatesh, V., Thatcher, S. M. B., & Patel, P. C. (2015). Folding under pressure or rising to the occasion? Perceived time pressure and the moderating role of team temporal leadership. *Academy of Management Journal*, 58(5), 1313–1333.
- McDermott, C. (1999). Managing radical product development in large manufacturing firms: A longitudinal study. *Journal of Operations Management*, 17(6), 631–644.
- Middleton, P., & Joyce, D. (2012). Lean software management: BBC worldwide case study. *IEEE Transactions on Engineering Management*, 59(1), 20–32.
- Molokken-Østfold, K., & Jørgensen, M. (2005). A comparison of software project overruns—Flexible versus sequential development models. *IEEE Transactions on Software Engineering*, 31(9), 754–766.
- Moore, J. E. (2000). One road to turnover: An examination of work exhaustion in technology professionals. *MIS Quarterly*, 24(1), 141–168.
- Nan, N., & Harter, D. E. (2009). Impact of budget and schedule pressure on software development cycle time and effort. *IEEE Transactions on Software Engineering*, 35(5), 624–637.
- Ohly, S., Sonnentag, S., & Pluntke, F. (2007). Routinization, work characteristics and their relationships with creative and proactive behaviors. *Journal of Organizational Behavior*, 27(3), 257–279.
- Oliva, R., & Sterman, J. D. (2001). Cutting corners and working overtime: Quality erosion in the service industry. *Management Science*, 47(7), 894–914.
- Pearsall, M., Ellis, A., & Stein, J. (2009). Coping with challenge and hindrance stressors in teams: Behavioral, cognitive, and affective outcomes. *Organizational Behavior and Human Decision Processes*, 109(1), 18–28.
- Perlow, L. A. (1999). The time famine: Toward a sociology of work time. *Administrative Science Quarterly*, 44, 57–81.
- Petersen, K., & Wohlin, C. (2010). The effect of moving from a plan-driven to an incremental software development approach with agile practices. *Empirical Software Engineering*, 15(6), 654–693.
- Ply, J. K., Moore, J. E., Williams, C. K., & Thatcher, J. B. (2012). IS employee attitudes and perceptions at varying levels of software process maturity. *MIS Quarterly*, 36(2), 601–624.
- Port, D., & Bui, T. (2009). Simulating mixed agile and plan-based requirements prioritization strategies: Proof-of-concept and

- practical implications. *European Journal of Information Systems*, 18(4), 317–331.
- Powell, F. D. (1987). *Study of a software development process dynamic model* (Technical Report MTR 10314). Bedford, MA: Mitre Corporation.
- Rahmandad, H., & Sterman, J. D. (2012). Reporting guidelines for simulation-based research in social sciences. *System Dynamics Review*, 28(4), 396–411.
- Ramesh, B., Mohan, K., & Cao, L. (2012). Ambidexterity in agile distributed development: An empirical investigation. *Information Systems Research*, 24(2), 323–339.
- Repenning, N. P., Goncalves, P., & Black, L. J. (2001). Systems dynamics—Past the tipping point: The persistence of firefighting in product development. *California Management Review*, 43(4), 44–63.
- Repenning, N. P., & Sterman, J. D. (2002). Capability traps and self-confirming attribution errors in the dynamics of process improvements. *Administrative Science Quarterly*, 47, 265–295.
- Rosso, B. D. (2014). Creativity and constraints: Exploring the role of constraints in the creative processes of research and development teams. *Organization Studies*, 35(4), 551–585.
- Sarker, S., & Sarker, S. (2009). Exploring agility in distributed information systems development teams: An interpretive study in an offshoring context. *Information Systems Research*, 20(3), 440–461.
- Schatz, B., & Abdelshafi, I. (2005). Primavera gets agile: A successful transition to agile development. *IEEE Software*, 22(3), 36–42.
- Schaufeli, W., Bakker, A., & Van Rhenen, W. (2009). How changes in job demands and resources predict burnout, work engagement, and sickness absenteeism. *Journal of Organizational Behavior*, 30(7), 893–917.
- Seers, A., & Woodruff, S. (1997). Temporal pacing in task forces: Group development or deadline pressure. *Journal of Management*, 23(2), 169–187.
- Sengupta, K., & Abdel-Hamid, T. K. (1993). Alternative conceptions of feedback in dynamic decision environments: An experimental investigation. *Management Science*, 39(4), 411–428.
- Sengupta, K., & Abdel-Hamid, T. K. (1996). The impact of unreliable information on the management of software projects: A dynamic decision perspective. *IEEE Transactions on Systems, Man, and Cybernetics*, 26(2), 177–189.
- Sengupta, K., Abdel-Hamid, T. K., & Van Wassenhove, L. (2008). The experience trap. *Harvard Business Review*, 86(2), 94–102.
- Seshadri, S., & Shapira, Z. (2001). Managerial allocation of time and effort: The effects of interruptions. *Management Science*, 47(5), 647–662.
- Sterman, J. D. (2000). *Business dynamics—Systems thinking and modeling for a complex world*. Boston, MA: Irwin McGraw-Hill.
- Tatikonda, M. V., & Rosenthal, S. R. (2000). Successful execution of product development projects: Balancing firmness and flexibility in the innovation process. *Journal of Operations Management*, 18(4), 401–425.
- Thaler, R. H. (1999). Mental accounting matters. *Journal of Behavioral Decision Making*, 12(3), 183–206.
- Thomke, S. H. (1998). Managing experimentation in the design of new products. *Management Science*, 44(6), 743–762.
- van Oorschot, K. E., Bertrand, J. W. M., & Rutte, C. G. (2005). Field studies into the dynamics of product development tasks. *International Journal of Operations and Production Management*, 25(8), 720–739.
- van Oorschot, K. E., Sengupta, K., Akkermans, H., & Van Wassenhove, L. (2010). Get fat fast: Surviving stage-gates in NPD. *Journal of Product Innovation Management*, 27(6), 828–839.
- Vigden, R., & Wang, X. (2009). Coevolving systems and the organization of agile software development. *Information Systems Research*, 20(3), 355–376.
- Wickens, C. D. (1984). Processing resources in attention. In R. Parasuraman & D. R. Davies (Eds.), *Varieties of attention*. Orlando, FL: Academic Press.
- Wiersma, E. (2007). Conditions that shape the learning curve: Factors that increase the ability and opportunity to learn. *Management Science*, 53(12), 1903–1915.
- Wu, Y., Ramachandran, K., & Krishnan, V. (2014). Managing cost salience and procrastination in projects: Compensation and team composition. *Production and Operations Management*, 23(8), 1299–1311.
- Zelkowitz, M., Wallace, D., & Binkley, D. (2003). Experimental validation of new software technology. In N. Juristo & A. Moreno (Eds.), *Lecture notes on empirical software engineering* (pp. 229–263). Singapore: World Scientific Publishing.
- Zellmer-Bruhn, M. E. (2003). Interruptive events and team knowledge acquisition. *Management Science*, 49(4), 514–528.

Author Biographies

Kim E. van Oorschot is Professor of Project Management at BI Norwegian Business School, Oslo, Norway. She received her PhD in Industrial Engineering from Eindhoven University of Technology, Eindhoven, the Netherlands. Her research interests include decision making, trade-offs, and tipping points in dynamically complex settings, such as new product development projects. She can be contacted at kim.v.oorschot@bi.no

Kishore Sengupta is Reader in Operations Management and Director of Executive Education at Cambridge Judge Business School, Cambridge, United Kingdom. He received his PhD in business from Case Western Reserve University, Cleveland, Ohio, USA. His research examines the role of managerial behaviors in the execution of complex projects and the use of information technology at individual, group, and organizational levels. He can be contacted at k.sengupta@jbs.cam.ac.uk

Luk N. Van Wassenhove is the Henry Ford Professor of Manufacturing at INSEAD Fontainebleau, France. He is a Fellow of POMS, EurOMA, M&SOM, and EURO Gold Medalist. His research interests include closed-loop supply chains and humanitarian operations. He received his PhD in industrial management from the Catholic University of Leuven, Belgium. His research is concerned with operational excellence, supply chain management, quality, continual improvement, and learning. He can be contacted at luk.van-wassenhove@insead.edu